

Advancing Intrinsic and Non-Intrinsic Bug Classification with NLP, Machine Learning, and Few-Shot Prompt Engineering

by

Pragya Bhandari

B.E., Kathmandu University, 2020

A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF

MASTER OF SCIENCE

in

THE COLLEGE OF GRADUATE STUDIES

(Computer Science)

THE UNIVERSITY OF BRITISH COLUMBIA

(Okanagan)

July 2024

© Pragya Bhandari, 2024

The following individuals certify that they have read, and recommend to the College of Graduate Studies for acceptance, a thesis/dissertation entitled:

ADVANCING INTRINSIC AND NON-INTRINSIC BUG CLASSIFICATION WITH NLP, MACHINE LEARNING, AND FEW-SHOT PROMPT ENGINEERING

submitted by PRAGYA BHANDARI in partial fulfilment of the requirements of the degree of Master of Science

Gema Rodríguez-Pérez, Irving K. Barber Faculty of Science
Supervisor

Fatemeh Fard, Irving K. Barber Faculty of Science
Supervisory Committee Member

Mohammad Khalad Hasan, Irving K. Barber Faculty of Science
Supervisory Committee Member

Anas Chaaban, School Of Engineering
University Examiner

Abstract

Bug classification is a pivotal approach in analyzing bugs, streamlining debugging processes, and facilitating analysis. Recent studies have shown that bugs can be categorized into intrinsic and extrinsic types. Intrinsic bugs, which are traceable to a cause in a project’s code, contrast with extrinsic bugs, which lack such traceability and are typically instigated by external failures affecting the project. This thesis presents the first ever automatic bug classification tool called BuggIn using Natural Language Process (NLP) and Machine Learning (ML) models to classify intrinsic and non-intrinsic bugs and also introduces the application of few-shot prompt engineering in bug classification tasks by implementing a novel tool using Large Language Model (LLM). We conduct four experiments: 1) implementing an automatic bug classifier using solely text from the bug reports and training NLP and ML models; 2) assessing non-textual bug report features’ effectiveness for ML model inputs; 3) replicating the BuggIn pipeline with both textual and non-textual features (i.e., source code and code review metrics); and 4) exploring alternative techniques utilizing low-resource datasets through few-shot prompt engineering using GPT-3. The results are promising while using a combination of features for the bug classification task yielding much better classification score as compared to using only textual or only non-textual features. Furthermore, the results for experiments involving LLM and few-shot prompt engineering show potential in bug classification tasks, demonstrating comparable scores as the ML-based classifiers with much fewer data records. Overall, this thesis bridges the gaps in the literature involving the classification of intrinsic and non-intrinsic bugs by enhancing the classification model and by paving the way for novel methods of solving the task. Moreover, the inclusion of methods like few-shot prompt engineering can be a good alternative to other methodologies that require large datasets that have remained prevalent in bug classification research so far.

Lay Summary

Software development entails the occurrence of issues is inevitable. These issues are known as software bugs and are notorious among developers for being hindrances to good-quality software. Software bugs have been researched extensively and in this thesis, we explore bug-classification to analyze two kinds of bugs i.e., intrinsic and extrinsic. Intrinsic bugs are caused by faulty lines of code in the project, whereas extrinsic bugs are caused by failures in external entities that the project depends on. Classifying bugs into these categories significantly improves bug prediction models as prior studies have shown. We have experimented with technologies in Artificial Intelligence (AI) to classify intrinsic and non-intrinsic bugs using the textual and non-textual information in bug reports. The results show promise in classifying bugs demonstrating comparative analysis of different methodologies and features, and bridging the literature gap involving bug classification into intrinsic and non-intrinsic by paving the way for novel methods.

Preface

This thesis is the original work by the author, Pragya Bhandari conducted under the guidance of Dr. Gema Rodríguez-Pérez. The research methodology used in the Chapter 6 of this work involved the use of Generative AI which has been elaborated in detail. All images included in this thesis are created by the author. The thesis includes the original work titled "BuggIn: Automatic Intrinsic Bugs Classification Model using NLP and ML" that has been published in PROMISE 2023: Proceedings of the 19th International Conference on Predictive Models and Data Analytics in Software Engineering, and includes the work in Chapter 4. Also the work in the Chapters 5 and 6 have been submitted to the journal named Empirical Software Engineering (EMSE).

Table of Contents

Abstract	iii
Lay Summary	iv
Preface	v
Table of Contents	vi
List of Tables	ix
List of Figures	x
Acknowledgements	xi
Dedication	xii
Chapter 1: Introduction	1
1.1 Contributions	4
Chapter 2: Background	6
2.1 Bug Tracking System	6
2.2 Version Control System	7
2.3 Openstack	7
2.4 Intrinsic and Non-Intrinsic Bugs	8
Chapter 3: Literature Review	11
3.1 Bug Classification	11
3.1.1 Bug or Non-Bug Categorization	11
3.1.2 Bug Prioritization	14
3.1.3 Bug Root Cause	15
3.2 GitHub Issue Labels Classification	15
3.3 Classification Using Large Language Models (LLMs)	16

TABLE OF CONTENTS

Chapter 4: Application of NLP and ML in Bug Classification	18
4.1 Introduction	18
4.2 Approach	19
4.2.1 Data Collection	19
4.2.2 Data Pre-Processing	21
4.2.3 Embedding Techniques	21
4.2.4 Class Imbalance	22
4.3 Experimental Setup	22
4.3.1 Machine Learning Models	22
4.3.2 Hyperparameters Selection	24
4.3.3 Evaluation Metrics	25
4.4 Experimental Results	26
4.4.1 RQ1: How does the performance of various NLP and ML techniques differ in accurately identifying intrinsic bugs from bug reports title?	27
4.4.2 RQ2: How does the performance of various NLP and ML techniques differ in accurately identifying intrinsic bugs from bug report description?	29
4.5 Discussion	31
4.5.1 Classifying Intrinsic Bugs from Bug Reports' Textual Information	31
4.5.2 Limitations and Opportunities	32
4.5.3 Implications	33
4.6 Threats to Validity	34
4.7 Conclusion and Future work	35
Chapter 5: Source Code Metrics for Bug Classification	37
5.1 Introduction	37
5.2 Methodology	38
5.2.1 Dataset	39
5.2.2 Feature Selection	39
5.2.3 Pre-Processing	41
5.2.4 Embeddings	41
5.2.5 Machine Learning Models	42
5.2.6 Optimization	43
5.3 Experimental Results	43
5.3.1 RQ1: What is the performance of classification models when exclusively utilizing non-textual features of bugs for categorizing intrinsic bugs	43

TABLE OF CONTENTS

5.3.2	RQ2: How does the model’s performance differ when combining textual and non-textual features of bug reports for categorizing intrinsic bugs?	44
5.4	Discussion	49
5.4.1	Classifying Intrinsic Bugs Using Non-Textual Features	49
5.4.2	Classifying Intrinsic Bugs Using Combination of Features	50
5.4.3	Implications	50
5.5	Threats to Validity	52
5.6	Conclusion and Future Work	53
Chapter 6: Few-Shot Prompt Engineering		55
6.1	Introduction	55
6.2	Methodology	56
6.3	Experimental Results	57
6.4	Discussion	62
6.5	Threats to Validity	63
6.6	Conclusion and Future Work	64
Chapter 7: Conclusion		65
Bibliography		66

List of Tables

Table 3.1	Summary of Software Bug Classification Approaches .	12
Table 4.1	Classification logic	20
Table 4.2	Results of experiments conducted with Title of Bug Reports. P=Precision, R=Recall	29
Table 4.3	Results of experiments conducted with Description of Bug Reports. P=Precision, R=Recall	31
Table 5.1	Final list of features in our experiments, originally described by McIntosh and Kamei [39]	40
Table 5.2	Results of experiments conducted with Non-Textual features of Bugs between Intrinsic and Non-Intrinsic (Extrinsic and Non-bug) classes	45
Table 5.3	Results of experiments conducted with Combined features of Bug Reports to classify Intrinsic and Non-Intrinsic classes using description text in Textual feature. P=Precision, R=Recall	47
Table 5.4	Results of experiments conducted with Combined features of Bug Reports to classify Intrinsic and Non-Intrinsic classes title text in Textual feature. P=Precision, R=Recall	48
Table 5.5	Results comparing the experiments done with only textual features and the experiments done using combined features i.e., textual and non-textual	51
Table 6.1	Results of Few-Shot Prompt Engineering to Classify Intrinsic and Non-Intrinsic Bugs	62

List of Figures

Figure 2.1	Bug report of an intrinsic bug	8
Figure 2.2	Bug report of an extrinsic bug	9
Figure 2.3	Bug report of a non-bug	10
Figure 4.1	General Experiment Setup of RQs	26
Figure 5.1	Experiment Setup of RQ1	44
Figure 5.2	Experiment Setup of RQ2	46
Figure 6.1	Experiment Setup of RQ3	59

Acknowledgements

I owe the success of this academic journey to my supervisor, Dr. Gema Rodríguez-Pérez, who has been a constant mentor and a support in all areas of my graduate life. She has not only given me constant guidance and supervision, but also provided me with the freedom and space for my creativity. Her patience and confidence in me helped me progress not only as a student but also as a person. I will forever be indebted to you Dr. Gema for helping me through this journey. I would also like to extend my deepest gratitude to Dr. Fatemeh Fard for her guidance and time as she helped give my research direction multiple times. Her support and encouragement have been pivotal in my growth. I would like to extend my sincere thanks to Wade Klaver, the System Administrator with Advanced Research Computing (ARC) at UBCO, who had helped me through various obstacles during the implementation phase of my experiments that needed ARC servers. Many thanks to Dr. Mohammad Khalad Hasan for his guidance as my committee member. I would like to extend my sincere thanks to Dr. Yves Lucet for his kindness, inclusivity, and generosity in welcoming me into his lab space. Also, I cannot begin to express my thanks to my lovely fiancé Ashutosh Chapagain, who helped me see through the errors and the implementation strategies of the later stages of the research experiments. His technical consultancy as well as support on both emotional and spiritual levels have had a great impact on making the overall research process much smoother. He has been someone who inspires me and motivates me to strive for more, while he stands strongly as someone I can lean on. I would like to acknowledge all my labmates, Manaal, Rohit, Aria, and Jeena for filling our lab meetings with an intellectual discourse that was respectful, inclusive, and enjoyable. I gratefully acknowledge Mohamad Khajezade who helped me brainstorm ideas to overcome the obstacles in our research. I had the great pleasure of working in the same lab space as Tanmaya and Meghana with whom work almost felt like play. I would also like to acknowledge the kindness and love of my amazing friends Paulina, Justin, Anmol, Eranga, Tenzin, Hammad, and Tanin. You all have been a family away from home and I appreciate all the beautiful memories we have shared.

Dedication

I would like to dedicate the completion of my Master's journey to my parents, Dhruba Raj Bhandari and Pramila Baral Bhandari, who have supported and encouraged me throughout my life. They were the ones who gave me the confidence to cross continents. My father would sit alongside me as I drafted my essays to apply to graduate schools, while my mother would assist me in collecting all the documents I might need for the admission and immigration processes. Not only have they been my support system during this phase of my life, but they have also built the foundation of my character and I am eternally grateful to them.

My grandfather, Bhim Pani Baral, will always remain in my heart and has been the greatest science teacher and storyteller that I have ever known. Although he is not with us anymore, I want to express my sincere gratitude for all his love for me. I wonder what he would have said to me on the day I completed my graduate studies. Buwa, I will always remember you in all the milestones of my life.

Chapter 1

Introduction

We are living in a world where technology has become so ubiquitous and weaved into our daily lives that it is inevitably bound to impact our lives in some way or the other. We use various kinds of software in our smartphones in the form of mobile applications, web applications, complex enterprise systems, banking systems etc. However, no matter how skilled the software developers are, software bugs are an unavoidable aspect of the development process as software developers are humans and susceptible to making mistakes. These bugs can seriously affect the functionality, security, and reliability of software systems and a lot of effort has been made to control and enhance the quality of software systems. Software is usually not a build-once project, instead, it can be perpetually improved and added upon as new requirements and user expectations are explored. Assuring the quality of software involves meeting both the user needs or requirements and the technical specifications of the software. However, there may be scenarios when either the user requirements or software specifications are not completely met, leading to unexpected behaviors and issues in the end product. These unexpected or unwanted tendencies of software are called bugs. Fixing bugs is also part of the software maintenance process and software maintenance accounts for 60–90% of software life-cycle costs [8]. Addressing bugs is particularly challenging as it requires isolating the cause, comprehending potentially outdated code and functionality, and resolving the issue without disrupting the rest of the software or product.

Throughout a software's lifecycle, numerous bugs are detected which can become hard to keep up with. Therefore, teams in the industry generally use a Bug Tracking System (BTS) to manage the bug reports, and hence keep a track of the bugs. Some widely used BTS are Github issues¹, and JIRA², Bugzilla³. BTS enables the storage and organization of bug reports that can be sorted and searched by various details making it easier to access by the relevant parties.

¹<https://github.com/features/issues>

²<https://www.atlassian.com/software/jira>

³<https://www.bugzilla.org/>

When first opened, a bug report generally contains important defining information about the bug such as a title of the bug report to summarize the bug, a proper description elaborating what the bug is, how it can be replicated, where it was caused or sometimes even why it may have been caused. The text present in bug reports play a crucial role in describing the bug and can be leveraged to analyse and classify bugs with the help of NLP techniques. Bug reports also include various non-textual information such as priority of the bug, date when the bug was reported. As the bug is further discussed and acted upon, the bug report might undergo changes in assignees, priority, estimated fix date, and the number of comments. There can be multiple comments in each bug report including new findings and developments in the bug fixing process. The bug report has multiple states that have specific terminology such that a bug report is opened when a bug is first detected and reported, then a bug is in progress when it is assigned and actively being fixed, resolved or fixed when the bug fixing changes have been completed, and finally it is closed once the bug is fixed. All of this information is tracked and recorded within each bug report, and a project might have multiple such bug reports.

Bugs are easier to mitigate once they are classified into different categories. For example, classifying bugs based on their severity can enable developers to better prioritize bugs. Furthermore, bug classification facilitates bug analysis for knowledge transfer about the software maintenance patterns in a project. A plethora of research has been dedicated to bug classification to ease the process of bug fixing. Researchers have used BTS to gather data for research related to software bugs [7, 33]. A plethora of research has been dedicated to bug classification problems [35, 47]. There are several research works dedicated to automatic classification of bugs using various categorization criteria such as classifying bugs and non-bugs [6, 23], GitHub issue labels [12, 31], and priority levels [3, 27] among others.

In order to fix bugs, developers need to know which lines of code are faulty or where the bug originated from. It has conventionally been assumed that all bugs originate from a developer’s faulty lines of code. Nevertheless, recent studies have revealed that this assumption does not always hold true [51, 52]. With the rising usage of external libraries, dependencies, and APIs in a project, there have been scenarios wherein the bugs are caused by a change in the environment where the software is used, or because requirements changed in an external library used by the project, or by an external change to the version control system (VCS) of the project [51, 52].

Consequently, bugs can be classified into two types based on the origin of their causation: intrinsic and extrinsic. Intrinsic bugs can be traced

back to a specific point within the VCS where the bug-causing change was made. Whereas, extrinsic bugs are instigated by changes that lie beyond the project’s domain and control, making it impossible to trace them back to a specific bug-inducing commit (BIC) in the VCS

Extrinsic and intrinsic bugs pose a significant challenge to the outcomes of bug prediction models that rely on learning from past BICs to forecast future instances as such models have consistently treated all bugs as intrinsic. For example, Just-In-Time (JIT) bug prediction models [34] enable the identification of potentially risky commits during the integration of code changes into the VCS. JIT models allow practitioners to allocate their testing resources effectively, focusing on reviewing and addressing the most critical commits while the changes are still fresh in the minds of the authors [34].

The accuracy of bug prediction models heavily depends on identifying the specific commit that introduced the bug. Previous studies have utilized algorithms such as SZZ [55], V-SZZ [9], or RA-SZZ [43] to create bug corpus datasets and link those bugs with their bug-inducing commits. Unfortunately, for extrinsic bugs, there is no way to identify the associated bug-inducing commit in the VCS [52]. Consequently, when employing these approaches, the dataset fed into bug prediction models becomes susceptible to noise, as extrinsic bugs may be mistakenly linked to inaccurate bug-inducing commits. Recent work demonstrated that excluding extrinsic bugs from the dataset and training the bug prediction model exclusively on intrinsic bugs significantly enhances its performance [53]. This highlights the importance of an efficient approach to classify bugs into intrinsic and extrinsic types to refine the dataset used to train the bug prediction models. This specific classification task acts as a preparation step of training bug prediction models.

However, before this thesis, the only approach to classify bugs as intrinsic or non-intrinsic required manually analyzing the bugs and their textual information [51, 52], which is a very labor-intensive task. Prior studies that explored bug classification problems have focused on various classification categories like bugs or non-bugs [35, 47], priority levels [7, 31], root causes [15, 44] and so on, but no research has been done on automatic bug classification into intrinsic and non-intrinsic categories. To bridge this gap and assist researchers and practitioners in curating reliable datasets to improve the performance of bug prediction models, we present the *first automated approach for the identification of intrinsic bugs*. As suggested in previous studies, we leverage not only the rich information contained within the natural language of bug reports [3, 15] but also the non-textual information [12, 24] such as source code metrics relevant to the bug fixing commit (BFC). The features

are coupled with advanced Natural Language Processing (NLP) techniques and Machine Learning (ML) algorithms, as previously done in various other bug classification tasks [23, 28], to enable the effective identification of intrinsic bugs. In addition to NLP and ML, our experiments include few-shot prompt engineering techniques to better address the issues caused by a low-resource dataset. To the best of our knowledge, there is no other research that has applied few-shot prompt engineering methods for bug classification problems yet.

1.1 Contributions

The contributions of this research are listed below in this section.

- The first classification that automatically categorizes reported bugs as intrinsic or non-intrinsic
- A detailed literature review of studies on bug classification tasks that helps researchers to identify models and approaches used for those specific tasks.
- A comprehensive experimental evaluation to assess the performance of various NLP and ML techniques in classifying bug reports as intrinsic or non-intrinsic.
- A tool called BuggIn [11], which is the *first ever* automatic bug classification to classify intrinsic and non-intrinsic bugs, explores the application of NLP and ML in bug classification in the context of intrinsic and non-intrinsic bugs. This study focuses on leveraging textual features to identify intrinsic bugs.
- A comprehensive experimental evaluation to assess the performance of various types of input features (textual and non-textual) in classifying intrinsic bug reports.
- The use of few-shot prompts engineering to classify bugs as intrinsic the first time.
- A replication package⁴ that can be used and adapted by the research community to replicate our study and apply it to other domains.

⁴<https://github.com/STASER-Lab/Bug-Classification-Project>

1.1. CONTRIBUTIONS

The Chapter 4 contains the work that was published in ACM under the 19th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE'23) Conference held in San Francisco, California. This paper won the Best Paper Award in the PROMISE'23 Conference. The technical contributions as part of this paper are given in the respective chapters. The content from Chapters 5 and 6 combined together formulate our submission to the special issue of the journal named Empirical Software Engineering (EMSE). The submission is currently under review and extends the work presented in Chapter 4.

Chapter 2

Background

In this chapter, we expand on the various concepts that are involved in the research to better understand the background of our research. This chapter contains information on bug tracking systems, version control systems, OpenStack⁵ ecosystem, and examples of intrinsic, extrinsic and non-bugs.

2.1 Bug Tracking System

The dataset needed for general bug analysis as well as bug classification tasks are sourced from bug tracking systems (BTSs). The dataset used in this thesis was collected from Launchpad⁶, which is the BTS for the specific projects in OpenStack ecosystem. Bug reports and the kinds of information entered in them can vary across the type of BTS used in a project. In Launchpad, we can classify the information into two main parts i.e., textual information that includes the natural language text inserted by developers and non-textual information that includes status, priority and other details relevant to the bug.

The textual information in bug reports are not only important in describing and summarizing the bug, but also in tracking the discussion among team members throughout the bug’s life cycle. The **textual** labels found in bug reports of Launchpad are: 1) Bug title, 2) Bug description, and 3) Comments.

The **Non-textual** information in bug reports includes details about the date the bug was reported, the individuals associated with the bug (such as the reporter, assignee, and commentators), the project in which the bug was detected, the bug’s status, and its importance or priority. The individual who reports a bug and writes the bug report is typically referred to as the “bug reporter”, while the person to whom the bug is assigned is known as the “bug assignee”. The status of a bug indicates the stage of the bug-fixing process. For instance, when a bug is first reported, its status is marked

⁵<https://www.openstack.org/>

⁶<https://bugs.launchpad.net/>

as “new”. Once a developer begins addressing the bug, its status changes to “in progress”. After the bug has been fixed, the status is updated to “confirmed”, indicating that the fix has been verified and is ready for release. Finally, when the bug fix is incorporated into the main version, the status is updated to “fix released”. The importance of a bug is categorized as critical, high, medium, or low. This classification helps in planning and prioritizing bug reports within a given timeline.

2.2 Version Control System

Software is typically released with a version number indicating the number of updates made to its functionality. Due to the dynamic nature of software and the continuous modification of code, developers, and managers utilize a Version Control System (VCS) to archive and track all changes and versions of the software. Version control software, such as Git⁷, facilitates this process by providing an interface for developers to access different versions of their software and view information about changes. To accomplish this, the VCS employs a concept called commits, which serve as checkpoints for changes. Developers usually make atomic changes, involving modifications to a few lines of code, and commit these changes after verifying their syntactical, semantic, and logical correctness. These commits act as checkpoints, allowing developers to revert to previous versions of their code when necessary. These commits store a plethora of information which can be termed as source code metrics.

Various source code metrics can provide information about a commit. These metrics are lines added and deleted, changes to subsystems, directories, files, and entropy, signifying the dispersion of changes across files. More details about the metrics can be found in Chapter 5. This information can become especially significant in the case of commits that are involved in the bug-fixing process. In the Chapter 5, we leverage the source-code metrics as the non-textual features in the automatic bug classification model.

2.3 Openstack

The dataset for this thesis contains the bug reports that belong to the projects in the OpenStack⁸ ecosystem. OpenStack is an open-source cloud computing platform that caters to the needs of public and private clouds. It

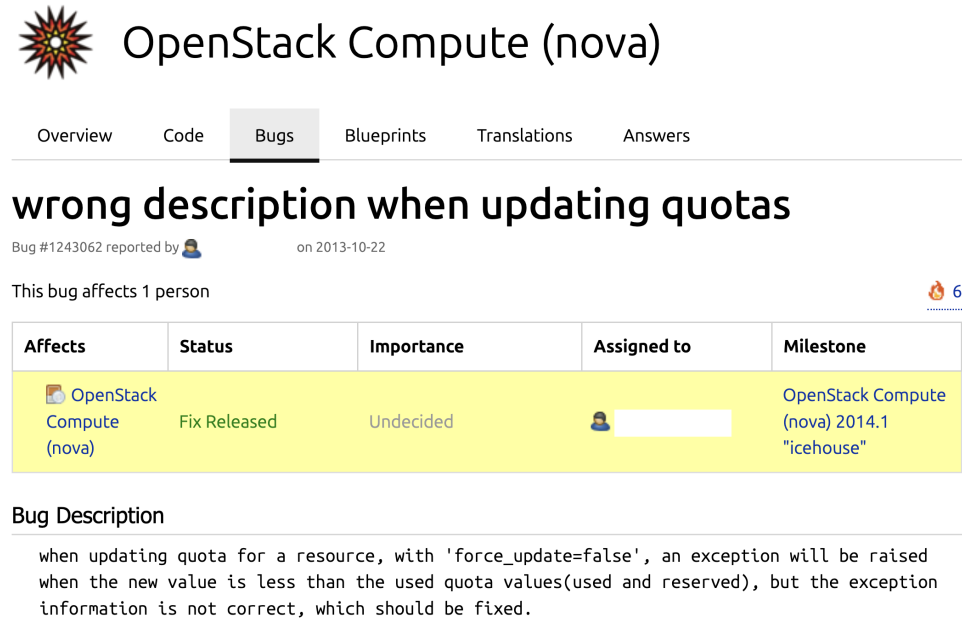
⁷<https://git-scm.com/>

⁸<https://www.openstack.org/>

provides an infrastructure-as-a-service functionality and controls large pools of compute, storage, and networking resources. OpenStack provides various services through its projects. In our dataset, there are bug reports belonging to 5 projects within OpenStack namely swift, glance, nova, neutron, and cinder.

2.4 Intrinsic and Non-Intrinsic Bugs

As defined earlier, intrinsic bugs are those that can be traced back to a BIC. Figure 2.1 reports that the exception information provided when a specific scenario happens is inaccurate and, therefore, requires correction. According to the definition of intrinsic bugs [52], the bug report in Figure 2.1 is clearly describing an intrinsic bug, as the bug originated in a change in the source code of the project. That source code was buggy and can be identified in the project's VCS.



The screenshot shows the OpenStack Compute (nova) bug report page. The title is "wrong description when updating quotas". It was reported by a user on 2013-10-22. The bug affects 1 person and has 6 votes. The bug is categorized as "Fix Released" with an importance of "Undecided". The assigned to field is empty. The milestone is "OpenStack Compute (nova) 2014.1 'icehouse'". The bug description states: "when updating quota for a resource, with 'force_update=false', an exception will be raised when the new value is less than the used quota values(used and reserved), but the exception information is not correct, which should be fixed."

Affects	Status	Importance	Assigned to	Milestone
OpenStack Compute (nova)	Fix Released	Undecided		OpenStack Compute (nova) 2014.1 "icehouse"

Bug Description

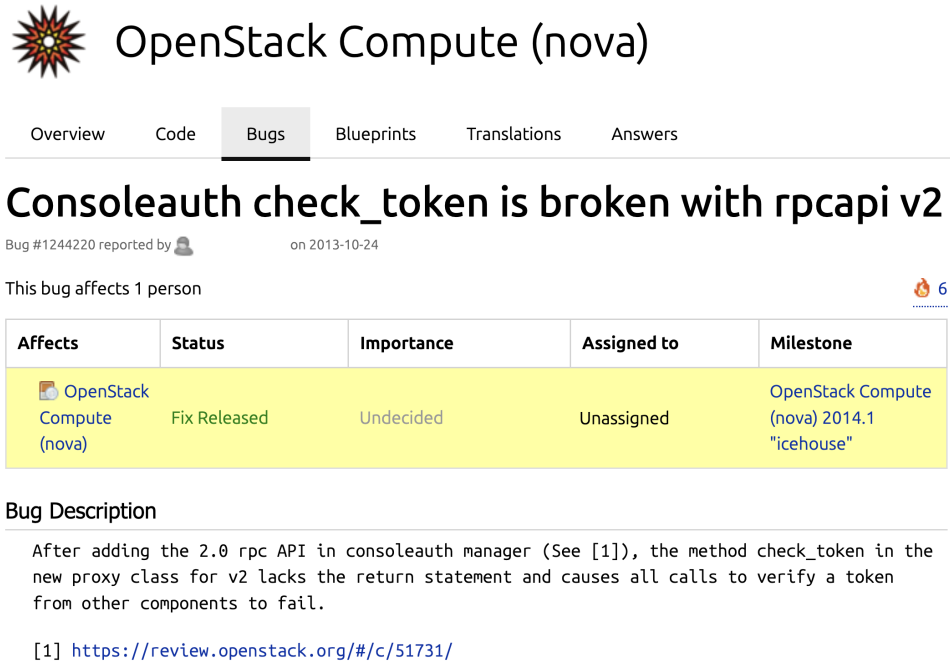
when updating quota for a resource, with 'force_update=false', an exception will be raised when the new value is less than the used quota values(used and reserved), but the exception information is not correct, which should be fixed.

Figure 2.1: Bug report of an intrinsic bug

Non-intrinsic bugs encompass two types of bugs, i.e., extrinsic and non-bugs. Extrinsic bugs cannot be traced to a BIC in the VCS. Figure 2.2 describes a bug in a method after incorporating the version 2.0 rpc API into the system. Consequently, this omission results in the failure of all

2.4. INTRINSIC AND NON-INTRINSIC BUGS

calls made by other components. According to the definition of extrinsic bugs [52], the bug report in Figure 2.2 is clearly describing an extrinsic bug, as the bug was not caused by buggy source code but by a change in a third-party API. That change cannot be found in Nova's VCS therefore, it is extrinsic to Nova.



The screenshot shows the OpenStack Compute (nova) bug report page. The title is 'Consoleauth check_token is broken with rpcapi v2'. It was reported by a user on 2013-10-24. The bug affects 1 person. A table lists the affected components: OpenStack Compute (nova) with status 'Fix Released', importance 'Undecided', assigned to 'Unassigned', and milestone 'OpenStack Compute (nova) 2014.1 "icehouse"'. The bug description states that after adding the 2.0 rpc API in consoleauth manager, the method check_token in the new proxy class for v2 lacks the return statement, causing all calls to verify a token from other components to fail. A reference [1] points to a review URL.

OpenStack Compute (nova)

Overview Code **Bugs** Blueprints Translations Answers

Consoleauth check_token is broken with rpcapi v2

Bug #1244220 reported by on 2013-10-24

This bug affects 1 person 6

Affects	Status	Importance	Assigned to	Milestone
OpenStack Compute (nova)	Fix Released	Undecided	Unassigned	OpenStack Compute (nova) 2014.1 "icehouse"

Bug Description

After adding the 2.0 rpc API in consoleauth manager (See [1]), the method check_token in the new proxy class for v2 lacks the return statement and causes all calls to verify a token from other components to fail.

[1] <https://review.openstack.org/#/c/51731/>

Figure 2.2: Bug report of an extrinsic bug

Non-bugs type of reports are those that were wrongfully reported as bugs and cannot also be traced to a BIC in the VCS of the project. For example when features are reported as issues, when a report describes a bug that may be caused in the future but has not occurred yet, or requests for code refactoring. Figure 2.3 describes a code redundancy that does not cause a failure or an issue in the project but highlights the need for code cleanup in the conditional statements. Therefore, this is a non-bug type of report that was wrongfully reported as a bug.

 neutron

OverviewCodeBugsBlueprintsTranslationsAnswers

neutron code redundancy

Bug #1265132 reported by  on 2013-12-31

This bug affects 1 person 

Affects	Status	Importance	Assigned to	Milestone
 neutron	Fix Released	Undecided		neutron 2014.1 "icehouse"

Bug Description

I found the code in l3_db.py is redundant,the code is below:

```
in function _update_router_gw_info:

    if gw_port and gw_port['network_id'] != network_id:
        fip_count = self.get_floatingips_count(context.elevated(),
                                                {'router_id': [router_id]})

    if fip_count:
        raise l3.RouterExternalGatewavInUseBvFloatinoId(
```

Figure 2.3: Bug report of a non-bug

Chapter 3

Literature Review

Several research studies have explored the potential of utilizing the content of bug reports to classify software issues and employed various methods tailored to different classification criteria. While a substantial portion of the existing literature on bug classification has primarily centered around determining whether a given bug report indeed pertains to a software problem or has been misclassified as an issue [2, 6, 23, 24, 35, 47, 57, 64], other studies have developed models for predicting the priority or severity level of bug reports [3, 7, 27, 31], pinpointing the root cause of bugs [3, 15, 44, 45], and forecasting GitHub issue labels such as “Bug”, “Enhancement”, or “Question” [12, 17, 29, 31, 33]. This section presents a literature review of the last nine years of research (from 2015 until 2024) that utilizes textual information from bug reports to classify software issues. With this literature review our goal is to inform researchers of the different models and approaches used by previous researchers when classifying bugs using textual information.

This chapter introduces the technologies other studies have used to classify bugs based on various categories. We also present the work that others have done in using few-shot prompt engineering related techniques in text classification. To the best of our knowledge, few-shot prompt engineering has not been used to solve bug classification problems yet.

3.1 Bug Classification

3.1.1 Bug or Non-Bug Categorization

Li et al. [35] employed bug titles and descriptions as input features for bug report classification in a prior investigation. They utilized Word2Vec to generate embeddings and implemented an Attention-based Bi-directional Long Short Term Memory (ABLSTM) model for the classification task, distinguishing bug reports as either bugs or non-bugs. Qin and Sun [47] used bug summary and description as input features. They compared the performance of a Long-Short Term Memory (LSTM) model with a softmax layer, with and without the integration of pre-trained word embeddings

3.1. BUG CLASSIFICATION

Table 3.1: Summary of Software Bug Classification Approaches

Paper	Year	Purpose	Approach
Li et al. [35]	2022	Bugs and Non-bugs	Word2vec and ABLSTM Ensemble method
Qin and Sun [47]	2018	Bugs and Non-bugs	LSTM with Softmax Layer
Zeng et al. [64]	2018	Bugs and Non-bugs	Cross-Project approach using SVM and LR
Afric et al. [2]	2023	Bugs and Non-bugs	RoBERTa
Terdchanakul et al. [57]	2017	Bugs and Non-bugs	n-gram IDF and LR
Fan et al. [24]	2017	Bugs and Non-bugs	Two staged classification using SVM and LR
Zhou et al. [65]	2016	Bugs and Non-bugs	MNB and Bayesian Net Classifier
Aktas et al. [6]	2023	Bugs and Non-bugs	n-grams and LinearSVC
Di Sorbo et al. [23]	2023	Safety related and Non-Safety related	TF-IDF+LSI+n-grams and RF
Alazzam et al. [7]	2020	Priority	Hybrid Embedding with RF and SVM
Izadi et al. [31]	2022	Priority	TF-IDF+SentiStrength ⁹ and RF
Ahmed et al. [3]	2021	Priority	TF-IDF and RF
Guo et al. [27]	2018	Severity	SVM
Catolino et al. [15]	2019	Bug Root Cause	TF-IDF and LR
Ahmed et al. [3]	2021	Bug Root Cause	RF
Kallis et al. [33]	2019	Github Issue Labels	n-grams and FastText [32]
Izadi et al. [31]	2022	Github Issue Labels	RoBERTa
Heo and Lee [29]	2023	Github Issue Labels	FastText [32]

3.1. BUG CLASSIFICATION

from Google [40]. This was juxtaposed against a Naive Bayes (NB) classifier combined with a topic-based Latent Dirichlet Approach (LDA) [46] and a Random Forest (RF) classifier combined with an n-gram Inverse Document Frequency (IDF) based approach [57]. Their findings suggest that the LSTM model with a final softmax layer, without using pre-trained word embeddings, yielded the most favorable results [47].

Zeng et al. [64] sought to classify issues as either bugs or non-bugs by contrasting a cross-project bug classification learning technique with a project-wise classification baseline. Their methodology involved employing concatenated bug titles and descriptions as input features. Support Vector Machine (SVM) and Logistic Regression (LR) were incorporated within their proposed ensemble modeling approach for cross-project-based learning. Their outcomes demonstrated the superiority of their proposed method in terms of F-score.

Afric et al. [2] worked on classifying issue reports as bugs or non-bugs, using bug titles and descriptions as input features. They conducted a comparative analysis of various approaches, including Simple Keyword Matching (KWM), Improved KWM (IKWM), FastText [32], and RoBERTa [36]. Experimental results disclosed the superior performance of RoBERTa in comparison to other classifiers. Likewise, Terdchanakul et al. [57] harnessed textual content from bug reports to classify bugs as either bugs or non-bugs. They conducted a comparative study involving two embedding techniques, namely N-gram IDF and LDA [46], in conjunction with two ML models, LR and RF. Their research highlighted that combining n-gram IDF embeddings with LR yielded the most promising results. Fan et al. [24] experimented with a two-staged classifier using two kinds of features, i.e., textual features from bug title and description, and developer features from GitHub such as the number of issues reported by the contributor, number of comments committed by the contributor, follower count, is part of core team or not, and so on. They used the standard preprocessing steps to process the textual features; the stopword removal approach was the technique that stood out. Fan et al. [24] retained some stopwords like “should,” “might,” and “not” to preserve the context of bug reports. They found that using TF-IDF with their proposed two-stage classifier involving SVM and LR using both textual and developer features yielded the best performance. Zhou et al. [65] also combined textual and non-textual features of bug reports to train their bug classification model better to categorize bugs from non-bugs. They used bug summary for textual features and priority, severity, component, assignee, and reporter as their non-textual features. They also implemented their model in two stages, using Multinomial Naive Bayes for

3.1. BUG CLASSIFICATION

the textual features and Bayesian Net Classifier for the non-textual features. Zhou et al. [65] report that their proposed approach yielded better results than previous studies done on the same dataset.

Aktas et al. [6] approached bug classification into issue or not-issue with bug reports written in Turkish. They reported that using TF-IDF, n-grams feature extractor, morphological analysis-based extractor, and pattern-based extractor combined with linear support vector classifier performed the best when compared with different feature extractors on their industrial dataset. Another paper that worked with a similar binary classification is that by Di Sorbo et al. [23] in which they aimed to classify issues into either safety-related or non-safety-related, which can be considered as a bug or non-bug situation for Unmanned Aerial Vehicles (UAV). They found the best results by employing the TF-IDF matrix, Latent Semantic Indexing (LSI), and n-grams for feature extraction and using Random Forest as the classifier. Their experiments involved other ML classifiers such as J48 Decision Tree, SVM with Sequential Minimal Optimization, etc., as well as FastText [32].

3.1.2 Bug Prioritization

Previous research has also explored utilizing textual information within bug reports to forecast bug priority through the construction of classification models. Alazzam et al. [7] use bug summary text in conjunction with non-textual attributes, including resolution status, product category, and the number of comments associated with bug reports. They introduced a hybrid embedding approach, amalgamating techniques such as Latent Dirichlet Allocation (LDA), Term-to-Term correlation, and neighborhood graph relations, subsequently evaluating diverse classifiers. Their investigations unveiled that the hybrid approach, when combined with RF and SVM, delivered the most promising results in addressing the bug priority classification challenge.

Izadi et al. [31] used the textual content of bug titles and descriptions to forecast bug priority. Their methodology leveraged TF-IDF alongside the natural language processing tool SentiStrength¹⁰. Their empirical findings revealed the superior performance of RoBERTa over alternative methods, including KNN, LR, MNB, and RF. Ahmed et al. [3] introduced a tool known as Capbug, which utilized bug summaries for bug priority classification. Their approach integrated TF-IDF, and they conducted a comparative analysis of various machine learning models, encompassing NB, RF, DT, and

¹⁰<http://sentistrength.wlv.ac.uk/>

LR. Their reported results highlighted the superior performance of RF when considering textual features.

Similarly, Guo et al. [27] conducted an investigation focused on classifying bugs according to their severity levels, employing bug summaries as the textual input. They conducted a comparative assessment involving a range of machine learning models, including NB, MNB, SVM, KNN, DT, and Random Tree. Their research demonstrated the superior performance of SVM compared to other models.

3.1.3 Bug Root Cause

A considerable body of literature has also explored automatic bug classification based on the bugs' root causes. Catolino et al. [15] utilized bug summaries to compare three embedding techniques, i.e., TF-IDF, Word2Vec, and Doc2Vec, and experimented with multiple ML models like NB, SVM, LR, and RF. Catolino et al. [15] found that the combination of TF-IDF and LR demonstrated the best performance. Ahmed et al. [3] used their tool CapBug for bug categorization based on root causes and found that textual features worked best with RF and TF-IDF.

3.2 GitHub Issue Labels Classification

A vast amount of research has utilized the dataset sourced from GitHub to classify bugs into one of the labels available in GitHub, such as bug, enhancement, question, documentation/support, etc. Kallis et al. [33] used n-grams vectorization of issue title and body text with FastText [32] to classify GitHub issues into bugs, enhancements, or questions. Similarly, for the same labels, Colavito et al. [17] experimented with fine-tuned variants of BERT and reported that the best performance yielded from RoBERTa when trained with the textual features of issues which are issue title and body. Another similar research was done by Izadi et al. [31] to classify GitHub issues into bug, enhancement, or support. They compared RoBERTa, FastText, Intention mining CNN, Bidirectional LSTM, MNB, and keyword-based rule approach and reported that RoBERTa outperformed them in the issue classification task. Heo and Lee [29] classified GitHub issues into 9 default labels, i.e., bug, enhancement, question, invalid, help-wanted, duplicate, wontfix, documentation, and good first-issue. They used SentenceBERT [49] to reduce label redundancies and experimented with FastText [32].

3.3 Classification Using Large Language Models (LLMs)

Recent advancements in the field of natural language processing (NLP) have been marked by the emergence of large language models, which have displayed considerable potential owing to their training on vast datasets and the possession of transferable base knowledge. As such, these models have come to be highly used in the discipline. One popular LLM is Generative Pre-trained Transformers (GPT). Its variant, GPT-3.5, represents a significant advancement in NLP and has shown strong performance in solving various NLP tasks in the few-shot setting [14].

The study conducted by Thaminkaew et al. [58] presents an innovative framework named PLAML (Prompt-Based Label-Aware Framework for Few-Shot Multi-Label Text Classification), which addresses the complexities of multi-label text classification through few-shot learning. Utilizing pre-trained Language Models (PLMs) such as the base iterations of RoBERTa [36], Tagalog RoBERTa [19], and WangchanBERTa [38] for English, Tagalog, and Thai respectively, PLAML integrates three pivotal techniques: (i) a token weighting algorithm accounting for label correlations, (ii) a template for enriching training samples to imbue the process with label-awareness, and (iii) a dynamic threshold mechanism for refining label predictions. Across various datasets encompassing multiple languages, PLAML consistently outperformed alternative baseline methodologies in few-shot text classification endeavors.

Loukas et al. [37] corroborate the efficacy of generative language models (LMs) such as GPT-3.5 and GPT-4 over Masked Language Model (MLM) counterparts, particularly when presented with limited examples. In their investigation employing the Banking77 dataset for financial intent classification, the authors underscore the viability of in-context learning with conversational LMs as a straightforward solution for rapid and accurate outcomes in few-shot scenarios. Although constrained by maximum token limitations, they employ a 1-shot setting for GPT-3.5 and a 3-Shot setting for GPT-4. Notably, GPT-3.5 achieved a micro-F1 score of 75.2% in the 1-Shot configuration, while GPT-4 attained a micro-F1 score of 83.1% in the 3-shot configuration, surpassing the performance of the S-MPNet-v2 method¹¹ within their respective shot settings. Loukas et al. also mentioned the expenses of using the GPT models and reported a total expenditure of \$1600.

For our experiments, we utilized “GPT-3.5-turbo,” a variant distin-

¹¹<https://huggingface.co/sentence-transformers/all-mpnet-base-v2>

3.3. *CLASSIFICATION USING LARGE LANGUAGE MODELS (LLMs)*

guished by its remarkable scale, boasting a total of 154 billion parameters. This model stands out for its capability to process an exceptionally long context, accommodating up to 16,385 tokens, while being more affordable than the GPT-4 variants. To the best of our knowledge, no previous research explores bug classification using LLMs or, more specifically, GPT models. Our study presents a novel approach to software bug categorization applying LLMs.

Chapter 4

Application of NLP and ML in Bug Classification

This chapter elaborates on the work that has been published in The 19th ACM International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE’23). The primary part of the study is the implementation of a tool called BuggIn [11] that is the first ever automatic bug classifier into intrinsic and non-intrinsic bugs. More specifically, it presents experiments to demonstrate the effectiveness of NLP and ML techniques in classifying intrinsic and non-intrinsic bug types.

4.1 Introduction

This chapter will explore the outcomes of applying NLP and ML in classifying bugs into intrinsic and non-intrinsic types. We used a manually labeled dataset of bug reports and we designed a series of experiments that involve three-fold comparisons using the textual features from the title and description of bug reports, two embedding techniques (seBERT [62] and TF-IDF [61]), and five ML algorithms. Our goal is to assist researchers and practitioners in collecting reliable dataset that only contains intrinsic bugs, i.e., bugs for which a bug-inducing commit can be identified. To achieve our goal, this chapter evaluates the efficacy of different embedding techniques and ML models in identifying intrinsic bugs based on the textual information extracted from bug reports.

Our findings reveal interesting distinctions in the performance of seBERT and TF-IDF based on the characteristics of the text being analyzed. While seBERT obtained higher performance when processing larger and more verbose bug report descriptions, TF-IDF yields better results when handling concise bug report titles. Taking into account the broader perspective of the automatic bug classification model’s best results, we observe that utilizing bug report titles as independent variables, employing TF-IDF for embedding, and employing the Decision Tree classifier model lead to an

exceptional overall F1-score of 78%. However, it is worth noting that when examining the differences in F1-scores between seBERT and TF-IDF, we observe a maximum discrepancy of merely one point in both cases where either seBERT outperforms TF-IDF or vice versa. This indicates that both embedding techniques showcase comparable performance for our specific dataset.

The main contributions made in this chapter are:

- The first classification that automatically categorizes reported bugs as intrinsic, solely relying on the textual information provided.
- A comprehensive experimental evaluation to assess the performance of various NLP and ML techniques in classifying bug reports as intrinsic or non-intrinsic.
- A replication package¹² that can be used by the research community to replicate our study and validate our results.

The remainder of this chapter is organized as follows. Section 4.2 introduces the details of our approach. Section 4.3 describes the experimental setup used. Section 4.4 presents the results. Section 4.5 discusses the findings and implications, while Section 4.6 contains the threats to their validity. Finally, Section 4.7 draws conclusions and provides the link to our reproducibility package.

4.2 Approach

We now explain the approach we followed. It includes data collection, data pre-processing, embedding techniques, and class imbalance.

4.2.1 Data Collection

The dataset used in this research was sourced from the reproducibility package included in the study by Rodríguez-Pérez et al. [53]. The authors manually classified bug reports into intrinsic bug, extrinsic bug, and no-bug types. The replication package included a wealth of information for each bug report, such as the bug ID, the project name, and the presence of a bug-inducing commit. The dataset contains 1880 bug reports from OpenStack.¹³

¹²<https://zenodo.org/record/8125762>

¹³<https://www.openstack.org>

4.2. APPROACH

Table 4.1: Classification logic

Case	isBug	hasBIC	Classification
1	1	1	Intrinsic
2	1	0	Extrinsic
3	0	X	Non Bug

Despite the dataset’s comprehensive nature, it lacked crucial textual information about the bug’s description, which is necessary for our experimental analysis. Hence, we collected the textual information of bug reports from OpenStack issue tracking system (Launchpad¹⁴) using a bug-scraping script, which is included in our replication package. Then, we added this information to the dataset. Our extended dataset includes the following information:

- **Bug ID:** The unique identifier for each bug report.
- **Project:** The name of the project the bug belongs to.
- **isBUG:** Boolean values of 0 or 1 to represent whether the bug report was actually a bug or not based on manual data analysis.
- **hasBIC:** Boolean values of 0 or 1 to represent whether the bug report can be traced back to a Bug Inducing Commit (BIC) or not. This is the major indicator of whether the bug was an intrinsic bug or an extrinsic bug.
- **Description:** The entire bug description posted in the bug report.
- **Title:** The bug report’s title.

It is worth noting that in order to differentiate the classification of each bug report, boolean conditions utilizing the isBug and hasBIC values were employed to assign labels within the overall dataset. The two boolean values of isBug and hasBIC had been curated as part of the rigorous and manual data analysis performed by Rodríguez-Pérez et al. [53]. The classification logic is presented in Table 4.1. When the value of isBug is 1 and hasBIC is also 1, it indicates an intrinsic bug, whereas any other combination denotes a non-intrinsic type. Following the application of this labeling logic, out of

¹⁴<https://launchpad.net/openstack>

a total of 1880 bug reports, the dataset contained 1120 intrinsic bugs and 760 non-intrinsic bugs (212 extrinsic bugs and 548 non-bugs). We grouped extrinsic and non-bugs together to form the category “non-intrinsic” as our aim is to identify intrinsic bugs.

4.2.2 Data Pre-Processing

The main data used in our experiments consist of the textual information found in bug titles and descriptions. Upon analysis, we observed that the text comprised natural language containing details on bug replication and bug locations, snippets of faulty code, an assortment of technical and project-specific terminology, and lengthy error logs. In order to work with more meaningful text, we applied a comprehensive data cleanup process. This involved the removal of traceback error log data, alphanumeric SHA codes, and commit IDs, as well as special characters and numerical values. URLs and hyperlinks were replaced with the string “<URL>” to signify the presence of a URL. All characters were converted to lowercase, and project names were replaced with either “<internal project>” or “<external project>” depending on whether the bug belonged to the respective project. Stop-words were subsequently eliminated using the Gensim library’s¹⁵ standard functionality. Finally, we applied tokenization to transform each segment of text into an iterable list of tokens, and lemmatization to reduce each word to its fundamental root form.

4.2.3 Embedding Techniques

In our study, we generated word embeddings using the seBERT model [62] and the widely adopted TF-IDF method. SeBERT [62] model was proposed as a software engineering (SE)-oriented variant of the BERT model [22], specifically pre-trained on a large-scale SE dataset for contextual relevance [62]. Von der Mosel et al. [62] employed 119.7 Gigabytes of processed textual data extracted from diverse sources such as StackOverflow posts, GitHub issues and commit messages, and JIRA issues to create the SE-context-based dataset used for pretraining BERT. In addition to seBERT, we also employed TF-IDF as an alternative embedding technique for comparison. TF-IDF, which stands for Term Frequency-Inverse Document Frequency, has been extensively utilized in previous bug classification studies [3, 7, 15, 24, 31, 45, 64]. It quantifies the importance of a word within a

¹⁵<https://pypi.org/project/gensim/>

document collection or corpus based on its frequency in the document and its frequency across other documents.

4.2.4 Class Imbalance

In order to address the issue of class imbalance within the dataset, specifically the disproportionate representation of intrinsic (1120) and non-intrinsic (760) bug classes, we employed the Synthetic Minority Oversampling Technique (SMOTE) [16]. SMOTE is an effective technique for addressing class imbalance in machine learning tasks and has been used in previous studies on bug classification to handle class imbalance [3, 31]. It generates synthetic examples by interpolating between neighboring instances in the feature space. This approach allows for the creation of diverse and representative synthetic examples, effectively expanding the minority class and promoting a more balanced training set. Consequently, SMOTE helps in mitigating the risk of the classifier being biased towards the majority class and improves its ability to correctly classify minority class instances.

4.3 Experimental Setup

In this section, we first introduce the ML models studied. Then, we present the key parameter settings of our models. Finally, we describe the evaluation metrics we adopt.

4.3.1 Machine Learning Models

In this study, we employed a selection of five well-established machine learning models. Previous research have approached the bug classification task using similar ML models [3, 7, 15, 23, 24, 28]. Additionally, using simple ML models could be a better choice than other more complex Deep Learning or Transformer based classifiers due to the low-resource nature of our dataset. We implemented those models using the open source data analysis library scikit-learn library¹⁶ in Python.

Support Vector Machine (SVM) [18]: SVM is a supervised learning algorithm used for classification and regression tasks. It constructs a hyperplane or a set of hyperplanes in a high-dimensional feature space to separate different classes. SVM aims to maximize the margin between classes, making it effective for handling both linearly separable and non-linearly separable

¹⁶[urlhttps://scikit-learn.org/stable/supervised_learning.html](https://scikit-learn.org/stable/supervised_learning.html)

datasets.

Random Forest [13]: It is an ensemble learning method that combines multiple decision trees to make predictions. It creates a collection of decision trees by randomly selecting subsets of features and training each tree independently. During prediction, the final result is determined by aggregating the predictions of individual trees. Random Forest is known for its robustness against overfitting and its ability to handle high-dimensional datasets.

K-Nearest Neighbors (KNN) [20]: KNN is a non-parametric classification algorithm that assigns a new data point to the majority class of its nearest neighbors. It works by calculating the distances between the new data point and all existing data points in the training set. The value of “K” determines the number of neighbors considered. KNN is simple to implement and suitable for small to medium-sized datasets.

Decision Tree [48]: It is a flowchart-like tree structure where each internal node represents a feature or attribute, each branch represents a decision rule, and each leaf node represents the outcome or class label. It partitions the dataset based on the feature values to recursively build a tree-like structure.

Logistic Regression [1]: It is a statistical model used for binary classification. It models the relationship between the input features and the probability of belonging to a specific class using the logistic function. LR estimates the coefficients of the input features to make predictions. It is widely used due to its simplicity, interpretability, and ability to provide probabilistic predictions.

These models have been widely utilized within the research community for addressing similar problems [3, 7, 12, 15, 27, 31, 31, 33, 35, 44, 45, 47, 57, 64]. The independent variable in our analysis was the textual feature, specifically the bug report title or description, while the dependent variable corresponded to the bug class, either intrinsic or non-intrinsic. To optimize the performance of each model, we utilized the Grid Search optimization algorithm [10] for hyperparameter tuning. This approach allowed us to systematically explore various combinations of hyperparameters and select the optimal configuration for each model. The hyperparameter results are described in the next subsection. For robust evaluation and to account for potential biases, we employed the Stratified k-fold cross-validation method with a value of k set to 5. Stratified k-fold cross-validation is a widely used technique for evaluating the performance of machine learning models. In this approach, the original dataset is divided into k equally sized folds while maintaining the proportion of instances from each class in every fold. Specifically, stratification ensures that each fold contains a representative

distribution of the different classes present in the dataset. By doing so, it helps to mitigate the potential bias that may arise from imbalanced class distributions. In our study, we employed a value of $k=5$, meaning that the dataset was divided into five folds, allowing us to perform model training and evaluation five times, with each fold serving as a testing set once while the remaining folds were used for training.

4.3.2 Hyperparameters Selection

Each ML model was configured with a specific set of hyperparameters, some of which were common across models, while others were model-specific.

For SVM, the primary hyperparameter considered was the kernel type. SVM utilizes mathematical functions for modeling known as kernels, and we experimented with four types: linear, polynomial, radial basis function (RBF), and sigmoid. Additionally, we employed the class weight hyperparameter to assign custom weights to each class in the dataset. Class weights hyperparameter was employed to ensure that the model’s training process accounted for the imbalanced distribution of intrinsic and non-intrinsic bugs, enabling it to achieve a more robust and balanced performance across the different bug classes. We explored three possibilities for class weights: balanced, 0.6, and 0.4 for intrinsic and non-intrinsic bug classes and reverse weights of 0.4 and 0.6 for intrinsic and non-intrinsic bug classes. Furthermore, we adjusted the gamma hyperparameter, which defines the kernel coefficient for RBF, polynomial, and sigmoid kernels. Gamma determines the influence of a single training example, with low values indicating a wider influence range and high values indicating a closer influence range. We tested two values for gamma: auto and scale.

LR was configured with four hyperparameters. The inverse of regularization strength, denoted as C , was set to two values: 1.0 and 0.1. Similar to SVM, LR employed class weights with the same range of possibilities. The penalty hyperparameter imposed a penalty on the logistic model to reduce the influence of less contributive variables, and we experimented with two penalty options: l1 and l2. Two solvers, namely liblinear and lbfgs, were chosen for the optimization problem of LR. The liblinear solver utilizes a coordinate descent algorithm, while the lbfgs solver approximates the Broyden–Fletcher–Goldfarb–Shanno algorithm.

DT and RF shared several hyperparameters. The criterion hyperparameter determined how split impurity would be measured during the tree-building process in DT. We set the maximum depth of the tree, minimum samples required in leaf nodes (`min_samples_leaf`), and minimum samples

4.3. EXPERIMENTAL SETUP

required to split (`min_samples_split`) an internal node, all within the range of 1, 2, 3 for both DT and RF.

Lastly, for KNN, we adjusted three hyperparameters: metric, number of neighbors, and weights. The metric hyperparameter allowed us to define the distance metric for computing similarity between neighbors, and we considered “euclidean” and “manhattan” options. The number of neighbors determined the count of neighbors checked during the classification of a query record, with values ranging from 1 to 3. The weights hyperparameter defined the weight function used, with options including “uniform” and “distance”. The uniform weight function assigned equal weight to all points in the neighborhood, while the distance weight function assigned higher weight to nearby points and lower weight to farther points.

4.3.3 Evaluation Metrics

In order to comprehensively assess the performance of our model, we employed four widely recognized evaluation metrics: precision, recall, accuracy, and F1-score.

Precision measures the proportion of correctly predicted positive instances (true positives) out of all instances predicted as positive (true positives + false positives). It quantifies the model’s ability to avoid false positives and provides insight into the accuracy of positive predictions.

$$Precision = \frac{TP}{TP + FP} \quad (4.1)$$

Recall, also known as sensitivity or true positive rate, calculates the proportion of correctly predicted positive instances (true positives) out of all actual positive instances (true positives + false negatives). It indicates the model’s ability to identify all positive instances without missing any.

$$Recall = \frac{TP}{TP + FN} \quad (4.2)$$

Accuracy calculates the ratio of correctly classified instances (true positives + true negatives) to the total number of instances. It represents the overall correctness of the model’s predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4.3)$$

F1-score is the harmonic mean of precision and recall. It combines both metrics to provide a single value that balances precision and recall. The

4.4. EXPERIMENTAL RESULTS

F1-score is useful when there is an uneven class distribution and serves as a balanced measure of the model’s performance.

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall} = \frac{2 * TP}{2 * TP + FP + FN} \quad (4.4)$$

Each metric is represented as a value between 0 and 1, with higher scores indicating better performance.

In addition, we conducted an evaluation of the Area Under the ROC Curve (AUC-ROC) score for each of our experiments. The AUC-ROC score provides insights into the classification potential of our binary classification model. By examining the AUC-ROC score, we gain a comprehensive understanding of the model’s ability to discriminate between intrinsic and non-intrinsic bugs.

4.4 Experimental Results

In this section, we present the motivation, approach, and results for the following two research questions:

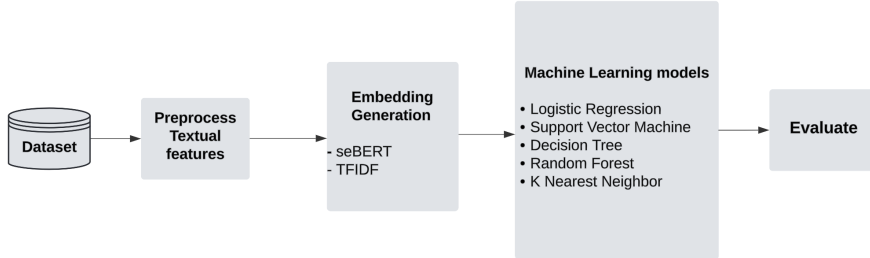


Figure 4.1: General Experiment Setup of RQs

4.4.1 RQ1: How does the performance of various NLP and ML techniques differ in accurately identifying intrinsic bugs from bug reports title?

Motivation: Using just the title of a bug report instead of the body description for training a classification model using NLP techniques might offer several benefits. The title provides a concise summary of the bug’s key characteristics, simplifying the input data and reducing computational complexity. Processing only the title is also more efficient, enabling faster model training. Additionally, focusing on the title might help filter out noise from verbose descriptions or irrelevant details present in the bug report descriptions. However, it is essential to consider the trade-offs and limitations of using only the title, as the body description may provide more detailed information and contextual clues that might contribute to accurate bug classification.

Approach: The independent variable for RQ1 is the textual information found in the bug report titles. We applied the pre-processing procedures outlined in the approach section to the independent variable. The dependent variable, indicating the bug class label (intrinsic or non-intrinsic), was transformed into boolean values, with intrinsic bugs assigned a label of 1 and non-intrinsic bugs assigned a label of 0. Following this, we employed our selected embedding techniques (seBERT and TF-IDF) to encode the independent variable. To ensure the availability of training and testing data, we performed an 80-20 split on the embedded data. Subsequently, we applied SMOTE to deal with the class imbalance in the training data. Finally, each ML model was trained using the Grid Search algorithm, which explored all possible combinations of pre-defined hyperparameters for each model. The training process employed a stratified k-fold cross-validation technique with k set to 5. To evaluate the performance of each model, we employed our evaluation metrics: precision, recall, F1-score, accuracy, and AUC-ROC.

Results: The experiments conducted with our implemented automatic bug classification model to categorize bugs into intrinsic and non-intrinsic categories using bug report title are presented in Table 4.2. We identified the optimal hyperparameter combinations for each ML model in our experiments. When utilizing seBERT embeddings, SVM achieved the best performance in terms of F1-score (77%) with the polynomial kernel, an automatic gamma value, and class weights of 0.6 for intrinsic bugs and 0.4 for non-intrinsic bugs. LR yielded the highest F1-score (73%) with a C value of 0.1, identical class weights as SVM, an l2 penalty, and the “liblinear” solver. DT exhibited its best F1-score (67%) with the “gini” criterion, a maximum

4.4. EXPERIMENTAL RESULTS

tree depth of 2, a minimum number of samples in leaf nodes set to 1, and a minimum number of samples required to split an internal node set to 2. For RF, the best F1-score (71%) was obtained when the hyperparameters were a maximum tree depth and minimum samples in leaf nodes both set to 3, and a minimum number of samples required to split an internal node set to 2. KNN performed optimally (34%) with the Manhattan metric, a single neighbor value, and uniform weights as compared to the other hyperparameter values.

In contrast, when employing TF-IDF embeddings, SVM demonstrated its highest F1-score (74%) when using balanced class weights, an automatic gamma value, and the sigmoid kernel. LR achieved its best performance (74%) with a C value of 1.0, class weights of 0.6 for intrinsic bugs and 0.4 for non-intrinsic bugs, an l1 penalty, and the “liblinear” solver. DT achieved its optimal F1-score (78%) with the “gini” criterion, a maximum tree depth of 3, a minimum number of samples in leaf nodes set to 1, and a minimum number of samples required to split an internal node set to 2. RF performed best (77%) with a maximum tree depth of 3, a minimum number of samples in leaf nodes set to 1, and a minimum number of samples required to split an internal node set to 3. Similarly, KNN exhibited its best F1-score (64%) with the Euclidean metric, a single neighbor, and uniform weights.

In terms of the AUC-ROC, the experimental results utilizing seBERT embeddings reveal that SVM and RF achieve the highest scores with 66%. They are closely followed by DT with a score of 63%, LR with a score of 62%, and finally, KNN with a score of 56%. When employing TF-IDF embeddings, the experimental outcomes demonstrate that LR achieves the highest AUC-ROC score of 64%. It is followed closely by RF with a score of 63.8%, followed by SVM with a score of 63%. DT obtains a score of 61%, while KNN attains the lowest score of 58%.

The results show that for bug report titles collected from OpenStack when utilizing the seBERT embedding technique, SVM achieved the highest F1 score (77%) as well as AUC-ROC score (66%), sharing the same highest AUC-ROC score with RF. Among the experiments conducted using TF-IDF, DT achieved the highest F1 score (78%), slightly outperforming the highest of seBERT combined with SVM, which achieved 77%.

4.4. EXPERIMENTAL RESULTS

Table 4.2: Results of experiments conducted with Title of Bug Reports.
P=Precision, R=Recall

Models	seBERT				TF-IDF			
	P	R	F1 Score	AUC-ROC	P	R	F1 Score	AUC-ROC
Support Vector Machine	71%	83%	77%	66%	69%	80%	74%	63%
Logistic Regression	68%	79%	73%	62%	70%	79%	74%	64%
Decision Tree	71%	63%	67%	63%	65%	96%	78%	61%
KNearest Neighbor	78%	22%	34%	56%	67%	61%	64%	58%
Random Forest	73%	70%	71%	66%	68%	89%	77%	63.8%

4.4.2 RQ2: How does the performance of various NLP and ML techniques differ in accurately identifying intrinsic bugs from bug report description?

Motivation: In light of the potential benefits associated with the detailed information and contextual clues present in bug descriptions, our study aims to investigate the performance of the approaches utilized in RQ1 when utilizing longer textual information from bug descriptions. Accurate identification of intrinsic bugs is crucial for software quality, as it helps to build reliable datasets to improve bug prediction models’ performance and increase their trustworthiness.

Approach: To answer RQ2, we use the description of bug reports as our independent variable. Then, we followed the same approach described in RQ1 and repeated all the experiments.

Results: The experiments conducted with our implemented automatic bug classification model to categorize bugs into intrinsic and non-intrinsic categories using bug report description are presented in Table 4.3. We identified the optimal hyperparameter combinations for each ML model in the experiments utilizing seBERT embeddings as well. LR yielded the best F1-score (76%) with a C value of 1.0, class weights of 0.6 for intrinsic bugs and 0.4 for non-intrinsic bugs, an l1 penalty, and the “liblinear” solver. SVM achieved its highest performance in terms of F1-score (75%) with the polynomial kernel, gamma value of “scale”, and class weights identical to LR.

4.4. EXPERIMENTAL RESULTS

For RF, the best F1-score (68%) was obtained when the hyperparameters were a maximum tree depth of 3 and minimum samples in leaf nodes set to 1, and a minimum number of samples required to split an internal node set to 2. DT exhibited its best performance (49%) with the “gini” criterion, a maximum tree depth of 3, a minimum number of samples in leaf nodes set to 1, and a minimum number of samples required to split an internal node set to 2. KNN performed optimally (11%) with the Manhattan metric, a single neighbor value, and uniform weights.

In contrast, when employing TF-IDF embeddings, LR again yielded the best F1-score (75%) with a C value of 1.0, balanced class weights, an l1 penalty, and the “liblinear” solver. Following LR closely, SVM achieved its highest performance in terms of F1-score (74%) with the sigmoid kernel, gamma value of “scale”, and class weights identical to LR. For RF, the best F1-score (67%) was obtained when the hyperparameters were a maximum tree depth of 3 and minimum samples in leaf nodes set to 1, and a minimum number of samples required to split an internal node set to 3. DT exhibited its best performance (37%) with the “entropy” criterion, a maximum tree depth of 3, a minimum number of samples in leaf nodes set to 3, and a minimum number of samples required to split an internal node set to 2. KNN performed optimally (35%) with the Euclidean metric, a single neighbor value, and uniform weights.

In terms of the AUC-ROC, the experimental results utilizing seBERT embeddings reveal that LR achieves the highest scores of 65.2%, which is closely followed by SVM with a score of 65%. RF exhibits an AUC-ROC score of 61.8%, DT scores 54.4%, and finally, KNN yielded a score of 51.2%. When employing TF-IDF embeddings, the experimental outcomes demonstrate the same order from best to worst AUC-ROC scores. LR achieves the highest AUC-ROC score of 66%, followed by SVM with a score of 65.4%, and then by RF with a score of 65.1%. DT obtains a score of 55.4%, while KNN attains the lowest AUC-ROC score of 52.6%.

The results indicate that for bug report descriptions collected from OpenStack when using seBERT, LR outperforms all other models in terms of F1-score (76%) as well as in terms of AUC-ROC score (65.2%). Also, when using TF-IDF, the highest scoring models remain the same, LR demonstrating the best F1-score (75%) and AUC-ROC score (66%).

4.5. DISCUSSION

Table 4.3: Results of experiments conducted with Description of Bug Reports. P=Precision, R=Recall

Models	seBERT				TF-IDF			
	P	R	F1 Score	AUC-ROC	P	R	F1 Score	AUC-ROC
Support Vector Machine	70%	81%	75%	65%	71%	77%	74%	65.4%
Logistic Regression	70%	83%	76%	65.2%	71%	78%	75%	66%
Decision Tree	66%	39%	49%	54.5%	72%	25%	37%	55.4%
KNearest Neighbor	72%	6%	11%	51.2%	65%	24%	35%	52.6%
Random Forest	70%	67%	68%	61.8%	74%	61%	67%	65.1%

4.5 Discussion

In this section, we discuss our findings, the limitations we found during our experimental setup, the opportunities to enhance our results in the future, and the implications of our results.

4.5.1 Classifying Intrinsic Bugs from Bug Reports’ Textual Information

At the beginning of our study, we posited that leveraging the knowledge of the textual information of bug reports could help develop approaches to automatically identify intrinsic bugs. Our results clearly show that bug reports can be classified into intrinsic and non-intrinsic bugs with a pretty high F1-score (76% and 77%) using machine learning models.

Catolino et al. [15] utilized bug summaries, which were summarized form of bug reports, to classify bugs based on root causes. They compared three embedding techniques: TF-IDF, Word2Vec, and Doc2Vec, and experimented with multiple ML models like Naive Bayes, SVM, LR and RF. They reported that the best performance (64%, in terms of F1-score) was obtained by using TF-IDF for embedding and LR as the classifier. Our study also indicates that combining TF-IDF and LR when using bug descriptions is the best approach for classifying intrinsic and non-intrinsic bugs, with an F1-score of 75%.

4.5. DISCUSSION

The implementation of DeepLabel by Li et al. [35] resulted in F1-scores of 79% and 89% for bug and non-bug classes. Their implementation utilized bug title and description as input, and combined Word2Vec, Attention-based Bi-directional LSTM (ABLSTM) model, and ensemble method to combine title classifier and description classifier together, yielding high results. In our study, we deliberately chose not to combine bug titles and descriptions to explore whether utilizing shorter textual information (bug titles) would yield comparable performance to the classification of intrinsic and non-intrinsic bugs. However, the combination of bug title and bug report is in our future research agenda.

Our findings indicate that using just the title of bug reports might be enough to identify intrinsic bugs, as indicated by the F1-score (78%) and AUC (65%). This is a very interesting finding as it implies that using longer text information does not outperform the classification of intrinsic bugs. We hypothesize that our findings might be due to the conciseness present in the title of bug reports. Titles often contain a succinct summary of the issue, providing a condensed representation of the bug’s key characteristics. Furthermore, bug report bodies usually can contain verbose descriptions, discussions, or irrelevant details that may introduce noise into the classification model. Focusing solely on the title can help filter out some of this noise. Thus, using only the title can simplify the input data and reduce the computational complexity of the classification model without losing performance. We suggest that if software practitioners have computational problems, they can use only the title of bug reports to classify bugs into intrinsic or non-intrinsic.

4.5.2 Limitations and Opportunities

Bug report features: Although our results indicate a favorable performance of the classification model using the textual information of bug reports (description or title), we have observed instances where its effectiveness is limited due to the insufficiency of words within bug reports for identifying intrinsic and non-intrinsic bug types accurately. Therefore, conducting further studies on the linguistic patterns employed by developers would be valuable in enhancing the classification of bug reports. We also hypothesize that some of the language used in bug reports might be context-dependent, which makes it more difficult for the approach to understand the context. Additionally, exploring different combinations of textual features through ensemble learning would be an opportunity for the future. While we have examined bug report titles and descriptions individually, there is potential

for enhanced performance by considering them in combination. Analyzing the value of other bug report features, such as developers’ discussions, or experimenting with other textual features, such as bug fixing commit messages, and exploring different combinations of these features could provide valuable insights to understand how different classification models perform when classifying intrinsic bugs.

Preprocessing techniques: In our study, following previous studies, we applied some pre-processing techniques. However, there is a need for future investigations to conduct a more comprehensive analysis to understand the specific impact of these pre-processing techniques on the performance of our classification model. To further advance the pre-processing stage, targeted techniques such as code abstraction and selective stopword removal can be considered. These techniques have the potential to refine the textual data by reducing noise and increasing the relevance of extracted features [12, 24, 35], ultimately aiming to improve the overall results of the classification model.

4.5.3 Implications

Bug prediction models: In this chapter, we have demonstrated that it is possible to identify intrinsic bugs using the textual information of bug reports. Our findings show that seBERT and TF-IDF can be effectively utilized for intrinsic bug identification. The highest performance scores (F1 score of 78%) were achieved by combining TF-IDF with the DT and utilizing the bug titles. This was closely followed by seBERT, SVM, and bug titles (with an F1 score of 77%).

By classifying intrinsic bugs and non-intrinsic bugs, our proposed approach aids not only JIT bug prediction models but all bug prediction models that use information from bug-inducing commits to train their models. Such models could achieve higher accuracy when trained solely on intrinsic bugs, providing a more faithful representation of real-world bugs. By incorporating our proposed approach as part of bug prediction practices, bug prediction models can improve their precision in detecting and forecasting intrinsic bugs, thereby enhancing their overall performance. Our approach does not mitigate the limitations of relying solely on the SZZ algorithm to link bugs with bug-inducing commits (i.e., false positives [25, 53], refactorings [43], and false negatives [50, 54]). However, previous studies have proposed solutions to control for this noise [21, 43]. Hence, we believe that by combining our approach with these previously proposed approaches, researchers and practitioners can obtain more reliable datasets that can be

used to feed bug prediction models.

Devising new tools: The process of curating bug datasets is resource-intensive, demanding significant effort and expertise in the specific software system. This labor-intensive task might incur substantial costs due to the expertise required, manual efforts involved, and data pre-processing, making it a financially demanding process. Thus the development of tools that help in the classification of bugs might be useful for researchers. In this chapter, we proposed an approach that can be integrated into the software engineering process as part of a software tool. This tool could automatically identify intrinsic bugs based on bug report titles or descriptions and flag them so researchers could automatically collect flagged intrinsic bugs when creating bug datasets.

4.6 Threats to Validity

The validity of this study is described in terms of the three main threats to validity that affect empirical software engineering research: construct, internal, external, and conclusion [63].

Construct Validity. Since we are using the replication package provided in Rodríguez-Pérez et al.’s paper [53], this study suffers from the same construct validity threats reported in Rodríguez-Pérez et al.’s study. From the dataset, 705 issues were analyzed by only a single rater. This might threaten the validity of the results. However, to minimize the impact of this threat, those raters were trained until they achieved a near-perfect agreement before they started classifying the 705 issues.

Internal Validity. The design of a classification model encompasses numerous subjective decisions, such as balancing the data, tuning the parameters, and validation techniques, among others. These decisions can potentially impact the classification results. To mitigate some of these threats, we follow the suggestions of previous studies and tune the hyperparameters [26, 56]. For validating the model, we relied on 5-fold cross-validation as our dataset is small, ensuring sufficient representation of different bug classes while maintaining computational efficiency. Finally, the domain-specificity of OpenStack might put some threats when using seBERT as seBERT has been pre-trained on the systems Github, Jira, and Stack Overflow. However, OpenStack uses Launchpad as its issue-tracking system, so there might be differences in the pattern of those systems that can affect the performance of seBERT in our study.

External Validity. The study of just one project, OpenStack, prevents

us from generalizing our findings to other systems. However, our goal was not to claim that our results would stand to all systems but rather to show that by using NLP and ML techniques, we can automatically identify intrinsic bugs. We think that our research is successful in this regard, as we demonstrate that intrinsic bugs can be identified using different techniques. We offer sufficient evidence that researchers and practitioners can use our proposed approach to identify intrinsic bugs to curate a dataset. Moreover, since OpenStack is considered a mature open-source project, results might not generalize to immature projects. Additionally, it is possible that other projects may require specific pre-processing steps due to the fact that some of these steps might depend on the domain-specific language of the project.

Conclusion Validity. We evaluate our classification model using a number of different evaluation metrics, such as precision, recall, F1, and AUC-ROC. It is known that AUC-ROC is sensitive to imbalanced data. Since our dataset is unbalanced, we used SMOTE to balance the data.

4.7 Conclusion and Future work

Currently, there is a lack of automated approaches to assist practitioners in the challenging task of classifying intrinsic and extrinsic bugs. Addressing this gap, our study aims to bridge this gap by introducing the first approach that classifies bug reports as either intrinsic or non-intrinsic (i.e., extrinsic and non-bugs) using their textual information. Furthermore, we conducted a comprehensive set of experiments to investigate the effectiveness of various combined techniques and explore the impact of the length of textual information on classification performance.

Our results indicate that both seBERT and TF-IDF are effective in identifying intrinsic bugs. The best performance was observed when combining TF-IDF with the Decision Tree algorithm and utilizing bug titles, resulting in an F1 score of 78%. Following closely, seBERT, Support Vector Machine, and bug titles achieved a strong F1 score of 77%. Furthermore, our findings suggest that incorporating longer textual information, specifically the bug reports' descriptions, does not yield superior performance compared to models utilizing shorter information, such as bug report titles.

Our future work focuses on improving the devised model and creating a taxonomy for intrinsic and non-intrinsic bugs. There are several potential avenues for further enhancing our classification approach as we have discussed in the limitations and opportunities subsection. First, the utilization of contemporary classifiers, such as Deep Learning models or fine-tuned

4.7. CONCLUSION AND FUTURE WORK

Large Language models could be explored to increase the performance of our classification model. These advanced techniques might yield improved performance when applied to our dataset. Second, it would be beneficial to conduct experiments and compare the effectiveness of non-textual features, such as source code metrics, in classifying bugs into intrinsic and non-intrinsic categories. By examining the efficiency of these features in comparison to textual features extracted from bug reports, we can determine if they offer substantial improvements in bug classification performance. Also, these experiments would help in creating the taxonomy of intrinsic and non-intrinsic bugs. Finally, we will explore the potential benefits of incorporating information regarding the origin of bugs, e.g., external dependencies associated with extrinsic bugs might contribute to their perceived importance in comparison to intrinsic bugs.

Replication Package: In order to promote reproducible research, the dataset and scripts for this chapter are available in our replication package.¹⁷

¹⁷<https://zenodo.org/record/8125762>

Chapter 5

Source Code Metrics for Bug Classification

In this chapter, we explore ways to improve the classification model presented in Chapter 4 with the same goal to classify intrinsic and non-intrinsic bugs by performing experiments to compare the performance of using different kinds of features, specifically non-textual features.

5.1 Introduction

The relevant information for addressing software bug classification primarily resides within bug reports and bug-fixing commit (BFC) data. Bug reports, usually, contain enough information about the bug lifecycle, using textual content to detail the bug’s nature, progression, and eventual resolution encapsulated within closing remarks specifying the BFC. Additionally, BFCs include insights into the changes done to fix the bug. For example, lines of code added, removed, or modified, or the number of modified files among others. Whereas bug report data is typically sourced from bug tracking systems, BFC data can readily be extracted from Version Control Systems (VCS). Alongside these types of information that have been utilized in bug classification tasks [7, 15, 23, 27, 33], several classification methods are more widely used than others [33, 65]. For instance, Natural Language Processing (NLP) techniques, Machine Learning (ML) techniques [23, 28], Deep Learning methods [5, 44], and novel algorithms [41, 64] have been prevalent in bug classification tasks.

In the prior chapter, we introduce BuggIn [11], the first bug classification model that classifies intrinsic bugs with an F1-score of 78%. BuggIn [11] relies on the utilization of textual features extracted from bug reports, such as titles and descriptions, combined with embedding techniques (i.e., like seBERT [62] and TF-IDF [61]), and conventional Machine Learning (ML) models. In this chapter, we extend our previous chapter with the goal to improve our bug classification model by exploring the efficacy of alternative

input features and approaches. Specifically, this chapter extends our previous chapter [11] by (1) adding a literature review on studies that have used bug report information to build classification models. (2) Examining non-textual features of the BFC associated with bug instances, hypothesizing the existence of discernible patterns in the source-code metrics used to fix bugs, as well as other BFC-related information. (3) Combining textual and non-textual features to enrich the dataset with additional bug-related information, hypothesizing that more textual information could provide more context about the bug’s origin and enhance the performance of ML models.

This chapter aims to enhance bug classification techniques specifically tailored for effectively distinguishing intrinsic bugs from non-intrinsic bugs. Hence, the research questions are as follows:

- RQ1: What is the performance of classification models when exclusively utilizing non-textual features of bugs for categorizing intrinsic bugs?
- RQ2: How does the model’s performance differ when combining textual and non-textual features of bug reports for categorizing intrinsic bugs?

Our findings indicate interesting insights into the efficacy of including non-textual features in bug classification models. While individual models trained solely on non-textual feature (F1 71% and AUC-ROC 60.1%) did not surpass those trained exclusively on textual features (F1 78% and AUC-ROC 64%), our results demonstrate a notable enhancement in performance when including both feature types (F1 77% and AUC-ROC 69.4%) when compared with BuggyIn [11] (F1 78% AUC-ROC 64%).

The remainder of this chapter is organized as follows. Section 5.2 introduces the details of our approach and the methodology used. Section 5.3 describes the experimental results obtained. Section 5.4 discusses the findings and implications, while Section 5.5 contains the threats to their validity. Finally, Section 5.6 concludes and adds insights into future work that could extend the research further.

5.2 Methodology

This section explains the methodologies employed to address each research question. It provides comprehensive insights into the dataset utilized

and outlines the techniques applied for its processing. Furthermore, it expands upon the technologies and tools utilized in the experimental procedures.

5.2.1 Dataset

Using the dataset explained in Chapter 4, we augmented it by adding contextual features that are non-textual in nature.

The dataset comprises several non-textual features representing the source-code metrics of the bug-fixing commit. The addition of textual features enhances the information conveyed by the dataset about each bug report starting from the reporting to the fixing of the bug. Both types of features, textual and non-textual, have been leveraged in this chapter in varying sets of experiments to answer the research questions formulated above. While the features make up the independent variables in this chapter, the dependent variable is the class of the bug, which we also refer to as “labels” in this chapter.

The non-textual metrics were extracted from [39] and included in our dataset. These non-textual metrics are related to size changes, historical aspects, author and reviewer experience, and the review process. Size-related metrics include the number of lines added and deleted, modifications to subsystems, directories, files, and entropy, representing the spread of changes across files. Historical aspects are captured through the count of unique prior changes, the involvement of developers over time, and the age of the changes. Author and reviewer experience metrics involve the number and recentness of prior changes an actor has participated in and their engagement in subsystem changes. The review process is characterized by the number of iterations, reviewers, comments, and the time taken for review and approval. This taxonomy of non-textual metrics of bugs provides a multifaceted perspective on the characteristics of software changes, thus facilitating a rounded understanding of the development and review processes.

5.2.2 Feature Selection

The feature selection process was done by performing Spearman correlation analysis among all the source code metrics. We removed one of two correlated features among all sets of features with correlations of 0.7 and higher as done in previous studies [39, 53]. After this step, the number of features was reduced from 30 to 14. Table 5.1 describes the final list of features.

5.2. METHODOLOGY

Table 5.1: Final list of features in our experiments, originally described by McIntosh and Kamei [39]

Property	Description	Acronym
BIC	The number of bugs the change introduced.	bugcount
BFC	The number of bugs the change fixed.	fixcount
Lines added	The number of lines added by the change.	la
Lines deleted	The number of lines deleted by the change.	ld
Subsystems	The number of modified subsystems.	ns
Files	The number of modified files.	nf
Entropy	The spread of modified lines across files.	ent
Developers	The number of developers who have modified the file in the past.	ndev
Age	Time interval between the last and the current changes.	age
Prior changes	The number of prior changes that an actor ¹⁸ has participated ¹⁹ in.	aexp
Awareness	The proportion of <i>aexp</i> to <i>ns</i> that an actor has participated in.	asawr
Iterations	Number of times that a change was revised before integration.	nrev
Review Window	The length of time between the creation of a request and its final approval for integration.	rtime

Moreover, the methodology to address RQ2 specifically incorporates feature reduction through Principal Component Analysis (PCA). PCA serves as a dimensionality reduction technique aimed at condensing the multitude of variables or features within a dataset while preserving essential information reflecting prevalent trends or patterns. This process effectively mitigates the computational complexity inherent in training ML models and acts as a preventative measure against overfitting. More specifically, PCA reduced the complexity due to the large dimensional features generated by the seBERT embedding technique, thus ensuring a more optimal pipeline.

5.2.3 Pre-Processing

The features utilized in this chapter were of two kinds - textual and non-textual. The approaches used for pre-processing these kinds of data differ significantly as they require different data handling and cleaning techniques. For the textual features, the pre-processing steps have remained identical to that in the pipeline mentioned in the prior chapter BuggIn [11] such as removing error logs, commit IDs and special characters, replacing URLs with “< URL >”, stopword removal, lowercasing all letters, and tokenization among others. The pre-processing of the textual features involved the processing of bug report text content found in the report title and description. For the non-textual features, we normalized the data, using min-max normalization, to mitigate the potential impact of varying scales among different features. This step also ensures that each feature contributes proportionally to the model training process.

5.2.4 Embeddings

The experiments conducted in this research involve two main embedding techniques: seBERT [62] and TF-IDF. These embedding techniques have been used to process the textual features of bug reports, like the bug title and bug description. SeBERT [62] is a transformer-based embedding method. As the name suggests, it is a BERT model that has been pre-trained in software engineering (SE) data. The training of seBERT involved utilizing 119.7 gigabytes of processed textual data mined from various sources, including StackOverflow posts, GitHub issues, commit messages, and JIRA issues. The paper by Von der Mosel et al. [62] found that transformer models trained on general domain data struggle to comprehend the SE domain text, whereas a model trained specifically on SE data can not only understand the context better but also overcome the obstacle caused by the

overlapping vocabulary between general domain and SE domain. Although seBERT was originally validated on the Masked Language Model (MLM) and sentiment analysis-related tasks, it can also be used to generate embeddings and be used for our training purposes. Previous studies in SE have also used seBERT [6, 60].

TF-IDF, an established embedding technique widely employed in numerous domains of NLP [3, 7, 15, 31]. TF-IDF operates on the foundational principle of utilizing word frequencies within a document and across the entire corpus to show the significance metrics of words within a given text. The term frequency component assigns higher importance to words that occur frequently within a document, suggesting their relevance to the document’s topic, while the inverse document frequency component penalizes terms that are ubiquitous across all documents in the corpus, thereby identifying terms that distinctly represent the content of a specific text while disregarding those that are common across the corpus. These principles are encapsulated in the vector embeddings generated by TF-IDF, offering a nuanced representation of textual content that emphasizes salient terms while filtering out non-essential ones.

5.2.5 Machine Learning Models

Following the methodology used in our previous publication BuggIn [11], we employed the classic set of ML models: LR, SVM, DT, RF, and KNN. While training the models for non-textual features, we also added Multinomial Naive Bayes (MNB) in the list of models because there was no limitation of negative values hindering the model training for non-textual features. For hyperparameter tuning, we used Grid Search [10] algorithm as recommended in the previous pipeline. Grid Search Optimization method enabled a systematic exploration of different combinations of hyperparameters, facilitating the selection of the most optimal configuration for each model. To ensure a thorough evaluation and address possible biases, we utilized the stratified k-fold cross-validation technique with a value of k set to 5. This method is widely employed [7, 15] in assessing the performance of machine learning models. In this approach, the original dataset is partitioned into k folds of equal size while maintaining the proportion of instances from each class in every fold. Stratification ensures that each fold represents a balanced distribution of the various classes in the dataset, helping mitigate potential bias resulting from imbalanced class distributions.

5.2.6 Optimization

Furthermore, parallel computing techniques and concurrent ML training methodologies were incorporated to reduce the computational demands of using seBERT embeddings, ML models and a large feature set. This enhancement was devised to optimize processing speed and mitigate the time required for training, ultimately improving the performance of the existing pipeline significantly.

5.3 Experimental Results

In this section, we present the experimental results obtained from a series of meticulously designed experiments aimed at addressing the research questions posed in this chapter. The findings presented here offer valuable insights into the performance and effectiveness of these methodologies in accurately classifying bugs, shedding light on their potential implications for enhancing bug classification techniques.

5.3.1 RQ1: What is the performance of classification models when exclusively utilizing non-textual features of bugs for categorizing intrinsic bugs

Motivation: The previous chapter [11] only used textual features to classify intrinsic and non-intrinsic bugs, Rodríguez-Pérez et al. [53] show that intrinsic and non-intrinsic bugs present statistically significant differences in some of source code metrics used to predict bugs. We hypothesize that non-textual attributes (source code metrics from the BFCs) can be used to build bug classification models to categorize intrinsic bugs. Thus, the first experiment aims to utilize the non-textual features of BFCs associated with each bug reported in the dataset to study the learnability and distinction between the two class types, intrinsic and non-intrinsic

Approach: To address the first research question we pre-processed the non-textual features, which included techniques such as min-max normalization and data scaling, to prepare them for training ML models. Using the non-textual features as independent variables and the bug label as the dependent variable, we trained a variety of ML models, including LR, SVM, DT, RF, KNN, and MNB. The experimental setup has been depicted in the Figure 5.1. Hyperparameter tuning was performed using the grid search algorithm.

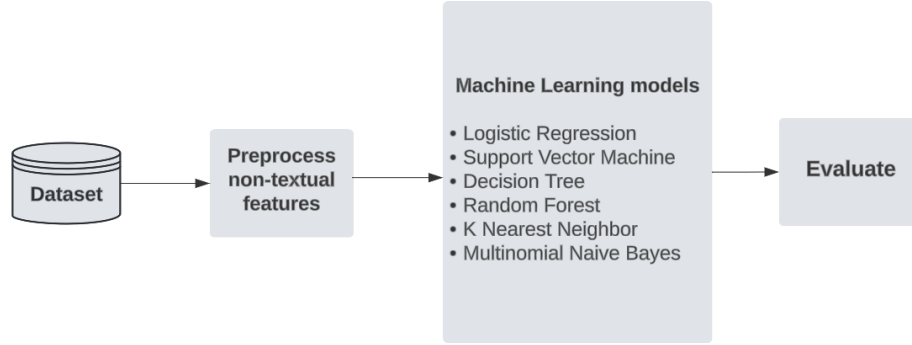


Figure 5.1: Experiment Setup of RQ1

Results: The results section presents the outcomes of classifying intrinsic and non-intrinsic bugs using only non-textual features.

Table 5.2 summarizes the results regarding precision, recall, F1 score, and AUC-ROC. Among the models evaluated, LR emerged as the top performer in classifying intrinsic and non-intrinsic bugs. LR exhibited commendable precision and recall scores of 60% and 70%, respectively, resulting in an F1 score of 69%. Additionally, LR demonstrated the highest AUC-ROC score of 60.1% among all the models, indicating superior categorising ability between the labels. Although MNB showed a much higher recall of 79% and the highest F1 score of 71%, it demonstrated much lower AUC-ROC score of 56.7% thus indicating an inferior minority class categorizing capability. This suggests that LR may be the most suitable model for this particular classification task, as it achieved a good balance between precision, F1, and AUC-ROC performance.

5.3.2 RQ2: How does the model’s performance differ when combining textual and non-textual features of bug reports for categorizing intrinsic bugs?

Motivation: While the RQ1 results do not surpass those of BuggIn [11], they exhibit a capacity to discern the minority class of non-intrinsic bugs to a certain extent. These findings prompted us to integrate both textual

5.3. EXPERIMENTAL RESULTS

Table 5.2: Results of experiments conducted with Non-Textual features of Bugs between Intrinsic and Non-Intrinsic (Extrinsic and Non-bug) classes

Models	Precision	Recall	F1 Score	AUC-ROC
Support Vector Machine	65%	69%	67%	57%
Logistic Regression	68%	70%	69%	60.1%
Decision Tree	65%	45%	53%	54.7%
KNearest Neighbor	61%	65%	63%	51.4%
Random Forest	67%	67%	67%	58.7%
Multinomial Naive Bayes	64%	79%	71%	56.7%

and non-textual features, thereby enriching the information available to the ML models. This approach aimed to ascertain whether the combined use of these diverse features could enhance classification performance by revealing underlying patterns in the labels. Hence, our experimental design encompassed the utilization of both textual features extracted from bug reports and non-textual features derived from BFCs linked to each bug report.

Approach: The approach employed in this chapter closely resembles that of RQ1, albeit with several adjustments to accommodate the substantial volume of features, stemming from the combination of different data types of independent variables. Specifically, the feature space is streamlined to 100 dimensions, optimizing the balance between computational efficiency and information retention.

The textual features used in this RQ are the title and the description of bug reports. The title provides a concise summary of the bug’s key characteristics, which can simplify the input data and reduce computational complexity. Focusing on the title might help filter out noise from verbose descriptions or irrelevant details present in the bug report descriptions. However, it is essential to consider the trade-offs and limitations of using only the title, as the body description may provide more detailed information and contextual clues that might contribute to accurate bug classification.

Hence, the first set of experiments was conducted to classify intrinsic and non-intrinsic bug types using the description text as the textual feature in combination with non-textual features. While the second set of experiments utilizes just the title text as the textual feature in the feature combination.

5.3. EXPERIMENTAL RESULTS

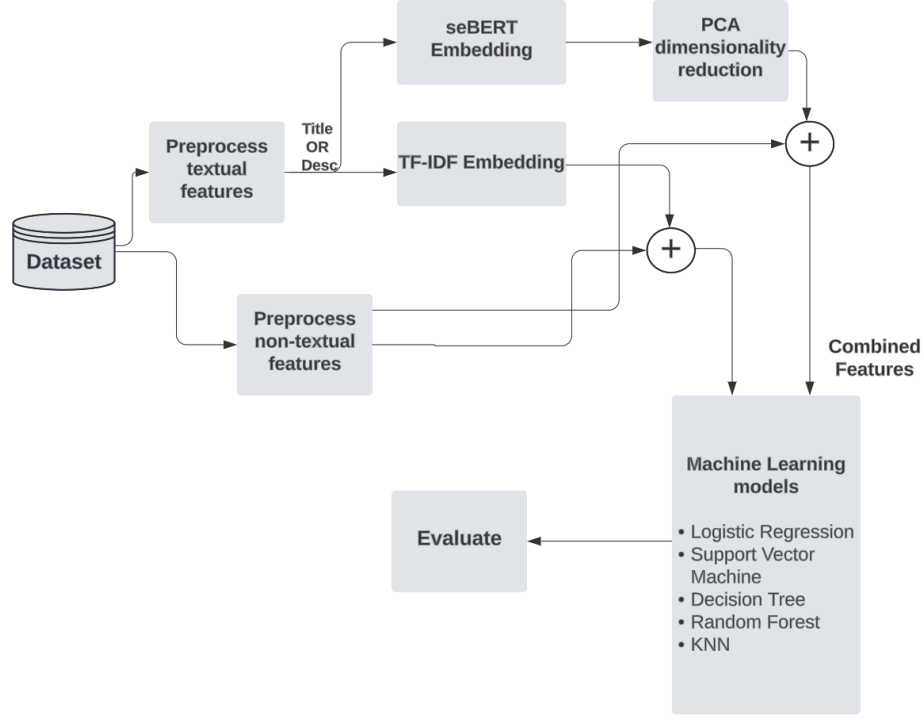


Figure 5.2: Experiment Setup of RQ2

Results: Both experiments compare two different embedding techniques seBERT and TF-IDF in addition to comparing the difference in performances when using the textual and non-textual features in combination. The embedding techniques were employed to process the textual feature, and evaluated the performance of various ML models.

seBERT + description text: Table 5.3 shows that for this experiment, SVM achieved a precision of 74%, with a recall of 78%, resulting in an F1 score of 76% and an AUC-ROC of 69.2%. SVM achieved the highest recall, F1 score and AUC-ROC score out of all other ML models, while yielding a comparable precision seconding with a one percent difference from the highest precision score of 75% in this experiment set. LR exhibited a precision of 75%, which is the highest compared to the other ML models, and an F1 score of 72%, with a slightly lower recall of 69%, leading to an AUC-ROC of 68.1%. DT demonstrated a precision of 63%, an F1 score of

5.3. EXPERIMENTAL RESULTS

Table 5.3: Results of experiments conducted with Combined features of Bug Reports to classify Intrinsic and Non-Intrinsic classes using description text in Textual feature. P=Precision, R=Recall

	seBERT				TF-IDF			
Models	P	R	F1 Score	AUC-ROC	P	R	F1 Score	AUC-ROC
SVM	74%	78%	76%	69.2%	73%	76%	75%	68.3%
LR	75%	69%	72%	68.1%	74%	75%	75%	68.5%
DT	63%	75%	68%	55%	68%	64%	66%	60.3%
KNN	70%	53%	60%	59.6%	66%	77%	71%	59.5%
RF	71%	67%	69%	63.8%	77%	61%	68%	67.7%

68% with a recall of 75%, and an AUC-ROC of 55%. KNN showcased a precision of 70%, a recall of 53%, and an F1 score of 60%, with an AUC-ROC of 59.6%. Finally, RF yielded a precision of 71%, a recall of 67%, an F1 score of 69%, and an AUC-ROC of 63.8%.

TF-IDF + description text: Table 5.3 shows that for this experiment, RF outperformed other models with a precision of 77% while achieving the third highest AUC-ROC score of 67.7%. RF exhibited a lower recall of 61% leading to an F1-score of 68%. SVM maintained competitive performance, achieving an F1 score of 75% and an AUC-ROC of 68.3%. LR demonstrated the same F1 score of 75% as SVM, which is the highest among other models in the experiments, and an AUC-ROC of 68.5%. DT achieved an F1 score of 66%, recall of 64%, and an AUC-ROC of 60.3%, while KNN yielded an F1 score of 71% and an AUC-ROC of 59.5%, meanwhile yielding the highest recall score of 77%. DT and KNN yield precision scores on the relatively lower end with precision scores at 68% and 66% as compared to the precision demonstrated by the other models.

In conclusion, among the ML models evaluated, SVM and LR consistently demonstrated robust performance across both embedding techniques when the textual content of description was used in combination with non-textual features during training. Specifically, SVM exhibited superior performance in terms of F1 score of 76% and AUC-ROC of 69.2% with seBERT, while LR showed competitive results with TF-IDF of 75% F1-score and 68.5% AUC-ROC. Additionally, RF emerged as the top performer with TF-IDF, showcasing its effectiveness in handling textual features for bug-type classification tasks.

seBERT + title text: Table 5.4 shows that SVM achieved a precision

5.3. EXPERIMENTAL RESULTS

Table 5.4: Results of experiments conducted with Combined features of Bug Reports to classify Intrinsic and Non-Intrinsic classes title text in Textual feature. P=Precision, R=Recall

	seBERT				TF-IDF			
Models	P	R	F1 Score	AUC-ROC	P	R	F1 Score	AUC-ROC
SVM	71%	75%	73%	66%	72%	74%	73%	66.8%
LR	70%	68%	69%	63.4%	74%	80%	77%	69.4%
DT	71%	61%	66%	62.3%	69%	60%	64%	60.7%
KNN	68%	49%	57%	58.3%	72%	62%	66%	63.6%
RF	70%	67%	69%	63.3%	69%	79%	74%	63.7%

of 71%, a recall of 75%, an F1 score of 73%, and an AUC-ROC of 66%, outperforming all other ML models in each of the scores. LR demonstrated competitive results with a precision of 70%, a recall of 68%, an F1 score of 69%, and an AUC-ROC of 63.4%. DT exhibited a precision of 71% identical to the highest precision score of SVM, however, DT achieved a lower recall score of 61%, an F1 score of 66%, and an AUC-ROC of 62.3%. KNN yielded a precision of 68%, a recall of 49%, an F1 score of 57%, and an AUC-ROC of 58.3% performing the worst among other ML models in this experiment setup. RF achieved a precision of 70%, a recall of 67%, an F1 score of 69%, and an AUC-ROC of 63.3%.

TF-IDF + title text: Table 5.4 shows that LR emerged as the top-performing model with a precision of 74%, a recall of 80%, an F1 score of 77%, and an AUC-ROC of 69.4%. SVM maintained competitive performance with a precision of 72%, a recall of 74%, an F1 score of 73%, and an AUC-ROC of 66.8%. DT exhibited a precision of 69%, a recall of 60%, an F1 score of 64%, and an AUC-ROC of 60.7%. KNN yielded a precision of 72%, a recall of 62%, an F1 score of 66%, and an AUC-ROC of 63.6%. RF achieved a precision of 69%, a recall of 79%, an F1 score of 74%, and an AUC-ROC of 63.7%.

LR demonstrated superior performance with TF-IDF, showcasing its efficacy in utilizing bug report title text combined with non-textual features for the bug classification task achieving an F1 score of 77% and AUC-ROC score of 69.4%. SVM also exhibited competitive results when paired with seBERT when using title text with an F1 score of 73% and AUC-ROC score of 66%.

5.4 Discussion

In this section, we discuss the findings and the implications of the results obtained in each research question.

5.4.1 Classifying Intrinsic Bugs Using Non-Textual Features

Our experiment aimed to develop an automated classifier capable of categorizing intrinsic bugs from the broader set of non-intrinsic types similar to the bug classes specified in BuggIn [11]. A discernible trade-off emerges between the F1 score and the AUC-ROC score, underscoring a complex interplay among model performance metrics. Notably, models exhibiting superior F1 scores do not necessarily excel in terms of AUC-ROC scores, and vice versa. For instance, Table 5.2 indicates the highest F1-score is attained by MNB 71%, with an AUC-ROC of 56.7%, while the highest AUC-ROC score is achieved by LR 60.1%, with an F1-Score of 69%.

This recurrent trade-off pattern can be attributed to the inherent challenge of binary classification problems, wherein models encounter difficulty achieving optimal performance across both classes. Specifically, when a model demonstrates proficiency in correctly predicting positive instances, thereby yielding a high F1 score, it may falter in accurately identifying negative instances, thus compromising its AUC-ROC score. Conversely, a model that excels in achieving a higher AUC-ROC score demonstrates adeptness in distinguishing between positive and negative instances, potentially at the expense of missing true positive instances, leading to a diminished F1 score within the positive class—in this context, the intrinsic bug class.

Moreover, the findings suggest that the grouping of extrinsic and non-bug types into a single non-intrinsic class may not be conducive when utilizing non-textual features. The inability to generalize and consolidate these classes underscores the importance of treating them as distinct entities, particularly when leveraging non-textual features for bug classification tasks.

The outcomes of RQ1 did not significantly improve the findings reported in BuggIn [11] (F1 78% and AUC-ROC 64%). Nevertheless, the use of non-textual features provides enough information to the model to discern between intrinsic and non-intrinsic bugs to a similar extent as textual features. These results support the observations by Rodríguez-Pérez et al. [53], who found that some code change properties, i.e., Author, History, Review, and Reviewer of the bug fixing commits linked to extrinsic bug reports and non-bug reports share some commonalities. Meaning that extrinsic bugs

have similar characteristics than non-buggy changes.

5.4.2 Classifying Intrinsic Bugs Using Combination of Features

When seBERT served as the embedding technique, BuggIn [11] achieved an F1 score of 77% using solely textual features, closely mirroring the results obtained from our RQ2 using combined features (both textual and non-textual features) with an F1 score of 76%. However, a more pronounced increase was observed in the AUC-ROC scores when employing combined features, registering a score of 69.2%, compared to the 66% AUC-ROC score yielded by the former approach. Similarly, with TF-IDF as the embedding technique, BuggIn [11] showcased the highest F1 score of 78%, while our proposed approach closely trailed with a 77% F1 score. Moreover, our approach substantially elevated the AUC-ROC score from the previous work’s peak of 66% to the proposed approach’s 69.4%. This underscores the superior capability of the second approach, employing combined features, to distinguish negative instances — non-intrinsic bugs in our context — compared to the methodology adopted in the prior work across both sets of experiments.

Furthermore, we observed differences in performance patterns concerning the type of text utilized in the experiments. BuggIn [11] excelled with title text in both embedding techniques, achieving F1 scores of 77% and 78% for seBERT and TF-IDF, respectively, while our proposed approach demonstrated superior performance when utilizing title text with TF-IDF, achieving an F1 score of 77%, and satisfactory performance when using description text with seBERT, resulting in an F1 score of 76%. Regardless of the embedding techniques employed, augmenting textual features with non-textual features of bugs enhanced the AUC-ROC scores significantly in both scenarios by enhancing the overall binary classification capability.

5.4.3 Implications

Trade-offs in Model Performance Metrics: Our results highlight the intricate trade-offs between different performance metrics, such as F1 score and AUC-ROC score. Researchers need to recognize these trade-offs when designing bug classification models. While a high F1 score indicates proficiency in positive instance prediction, it may come at the expense of AUC-ROC score, and vice versa. Understanding these trade-offs is crucial for the researchers to make informed decisions about their bug classification model. Depending on the specific goals and requirements of the software

5.4. DISCUSSION

Table 5.5: Results comparing the experiments done with only textual features and the experiments done using combined features i.e., textual and non-textual

Pipeline	seBERT		TF-IDF	
	F1 Score	AUC-ROC	F1 Score	AUC-ROC
BuggIn (Title) [11]	77%	66%	78%	64%
BuggIn (De- scription) [11]	76%	65%	75%	66%
RQ2 Results (Title)	73%	66%	77%	69.4%
RQ2 Results (Description)	76%	69.2%	75%	68.5%

project, they may prioritize either F1 score or AUC-ROC score, or seek a balance between the two. Our main goal to perform this classification was to identify non-intrinsic bugs from the training dataset of bug prediction models, hence it is more important not to predict non-intrinsic bugs as intrinsic than mislabeling intrinsic as non-intrinsic bugs. This implies, there is a greater importance to minimise false positives. Although in F1-score, the precision value takes the false positives into account by lowering precision values as the number of false positives increase, AUC-ROC score is better able to highlight the model’s performance by accounting for both true positive rate and false positive rate. The mathematical formulas of both terms are defined as

$$TPR = \frac{TP}{TP + FN} \quad (5.1)$$

, where TPR is true positive rate, TP stands for true positive, FN is false negatives, and

$$FPR = \frac{FP}{FP + TN} \quad (5.2)$$

where FPR is false positive rate, FP stands for false positives, and TN stands for true negative. Therefore, considering the importance of AUC-ROC score in achieving our overall goal, our model’s performance improves significantly when compared in terms of AUC-ROC score and the experiments performed with a combination of features outperforms those using only textual features.

Enhanced Classification with Combined Features: Bug classification is a complex task that often requires considering various aspects of software artifacts. Traditionally, bug classification models have relied heavily on textual features extracted from bug reports, such as titles, descriptions, and comments. While these textual features provide valuable information, they may not capture all relevant characteristics of bugs, especially for intrinsic and non-intrinsic bugs. Hence, by combining textual and non-textual features, researchers can create a more comprehensive representation of intrinsic bugs, capturing both their textual descriptions and their contextual relationships within the source code.

Devising new tools: Curating bug datasets is an inherently resource-intensive process. This process involves specialized knowledge of the software system being studied, as well as manual effort to collect and preprocess the data. As a result, it can be costly in terms of both time and expertise. Therefore, the development of tools to assist in bug classification shows potential for help researchers who face these challenges. Our results indicate that using both textual and non-textual data increases the performance of the model. Hence, researchers can use or enhance our proposed approaches to automatically identify and flag intrinsic bugs based on bug report titles, descriptions, and bug fixing commits linked to bug reports.

5.5 Threats to Validity

The validity of studies in empirical software engineering research can be elaborated using three main threats to validity: construct, internal, and external [63].

Construct Validity Since we have used the dataset labeled by Rodríguez-Pérez et al. [53], our study may have the same internal threats to validity as the previous study. There may be biases leading to mis-classification of bug reports thus influencing the experiments that learn from the examples in the dataset. As reported by Rodríguez-Pérez et al. [53], 705 issues were labeled by one person which might induce inaccuracies in our experiment, however, to mitigate the issue, the paper mentions that the raters underwent rigorous training sessions until reaching a level of agreement that approached near-perfection.

In addition to the dataset, the embedding techniques used in our study i.e., seBERT [62] and TF-IDF [61], have been used as provided by the libraries and resulting in inducing the same threats in construct validity in our study as reported in their implementation.

Internal Validity The dataset contains textual content that has been used in the experiments to train our models. The textual content was originally the bug title or description written by users or developers while reporting a bug. Often the textual content is not the most clear or representative of what the bug is actually about, thus possibly inducing some inaccuracies with the original ground truth that our study utilizes. However, in software engineering and the open source community, the real-world scenario would involve different types of reporters describing a bug while reporting it, thus making it a close representation of the nuances present in the actual field. The process of selecting non-textual features involved correlation analysis, which introduced a potential threat due to the necessity of discarding one of two pairs of correlated features. This introduced the risk of discarding a potentially important feature from the pair, thereby posing a threat to the integrity of the feature selection process and potentially impacting the overall classification performance.

External Validity The dataset contains the bug reports from the Open-Stack ecosystem of projects and may not be generalizable to other ecosystems as the projects, technologies, and technical jargon may differ across projects and ecosystems. This may have hindered the ability of ML models to isolate more generalized patterns helpful in classifying intrinsic and non-intrinsic bugs. Especially considering the correlation analysis of non-textual features that represent the source-code metrics of BFCs of bugs may vary across different ecosystems, thus significantly changing the overall pipeline and most inevitably the results. It is important to note that the dataset used in our studies may contain BFCs that are tangled. Tangled commits are the commits that solve more than one concerns [30]. Hence, if a BFC is a tangled commit, then it may contain code changes that have no relevance to the bug being fixed. The extra changes can be minor changes or code refactoring. However, the presence of tangled BFCs can affect the non-textual features as they include metrics such as number of lines of code added and modified, ultimately affecting the training process of ML models.

5.6 Conclusion and Future Work

In our previous chapter, we focused solely on textual information for bug classification. In this chapter, we explored different advanced classifiers and utilized a general-purpose large language model to boost performance. Our findings suggest that integrating non-textual features alongside textual ones can enhance bug classification accuracy. This chapter also highlights

5.6. CONCLUSION AND FUTURE WORK

the potential of utilizing few-shot prompt engineering methods to solve bug classification problems for low-resource dataset.

We observed an improvement in performance when both textual and non-textual features were employed in combination with advanced deep learning-based classifiers.

In the future, researchers can expand our bug classification research by including a wider range of datasets from different software ecosystems and projects, although acknowledging the challenge and cost associated with labeling these datasets for supervised learning. Researchers can also plan to experiment with various embedding models such as FastText, RoBERTa, and CodeBERT to compare their effectiveness (speed and performance) to our current embedding based approach.

While bug classification has garnered considerable attention in the literature, the presence of diverse classification criteria and the continual advancements in technology render it a perpetually engaging field of exploration. Despite significant progress in new technologies, there remains a substantial gap in the existing literature, presenting ample opportunities for further investigation. The outcomes of such research endeavors hold the potential to yield valuable insights that can benefit the entire software development industry. We contend that investigations aimed at classifying intrinsic and non-intrinsic bugs have broader implications wherein these techniques could serve as pivotal tools applicable and adaptable to the classification task of other bug types, offering valuable contributions to the field.

Chapter 6

Few-Shot Prompt Engineering

This chapter sheds light on the efficacy of exploring techniques alternative to the classic ML and DL approaches. We demonstrate the implementation of few-shot prompt engineering method and expand on the results obtained in the experiments shown in Chapter 5.

6.1 Introduction

We came across several obstacles in our attempts of automated bug classification into intrinsic and non-intrinsic. The lack of large labeled dataset, significant data imbalance between the labels, and the noise introduced by Tangled commits inhibiting the model training process. Therefore, it was evident that there was a need for a better approach to tackle the situation. One of the most trending topics in the past couple of years has been the rise of LLMs like GPT that have been rapidly integrated in various text classification research [37]. Also, several research demonstrated the efficacy of using few-shot learning to train models with satisfactory results [58]. We hypothesize that few-shot prompt engineering can also be applied to solve bug classification problem task. This study aims to answer the question: “To what extent does few-shot prompt engineering prove effective in classifying intrinsic bugs in the context of low-resource dataset?”

In this chapter, we extend our previous studies with the goal to explore the efficacy of non-classical approaches. Specifically, this chapter devises an alternative approach utilizing few-shot prompt engineering to address the same objective within the identical dataset, thereby evaluating the performance of a technique tailored for low-resource datasets like ours.

Our exploration of using an alternative methodology of few-shot prompt engineering showcased promising potential, achieving an F1 score of 73% and an AUC-ROC score of 62.58%. We recognize the need for further refinement and optimization of the few-shot prompt engineering approach, which holds

promise for surpassing traditional methods without necessitating extensive dataset requirements.

The rest of the chapter can be summarized into the following sections. Section 6.2 elaborates the pipeline used for prompt formulation and evaluation. Section 6.3 demonstrates the obtained results. Section 6.4 discusses the findings and implications, while Section 6.5 expands on the threats to their validity with respect to usage of LLMs. Finally, Section 6.6 concludes and adds insights into future work that could extend these experiments further.

6.2 Methodology

Few-shot prompt engineering is a method employed in scenarios with limited data resources, such as our case involving a dataset comprising only 1880 bug reports, which is insufficient for ML training and LLM fine-tuning. In such instances, few-shot learning offers a promising alternative to mitigate the need for extensive datasets, which can be resource-intensive. Prompt engineering, a technique utilized across various ML tasks, has also been explored with few-shot learning methods to address tasks traditionally requiring substantial datasets and computational resources [4, 42].

In few-shot learning, the term “shots” denotes the number of instances that accurately represent the dataset. In our binary classification study, one shot entails one example from each class, totaling two examples, while a four-shot experiment comprises four examples from each class, totaling eight examples. Prompt engineering involves issuing instructions or “prompts” to an LLM, such as a GPT, which generates responses that can be further analyzed or used as final solutions.

In our study, we use a few-shot prompt engineering pipeline leveraging the GPT-3.5 engine. Before initiating few-shot prompt engineering, the dataset underwent a 10-90% split, allocating 10% as the support set for guideline formulation and shot selection. We reserved the remaining 90% as the query set for unseen prompts to prevent data leakage. We adopted a 90-10 data partitioning strategy, allocating a substantially larger portion to the query set to rigorously assess the LLM’s performance. The context is established using a significantly lower number of example bug reports while having a large number of bug reports to test on, this split can simulate scenarios where the LLMs are expected to classify several bug reports generated over a period of time thus proving their robustness in solving bug classification tasks. We chose to set the temperature of the model to

6.3. EXPERIMENTAL RESULTS

0 in order to minimize the level of diversity or variability in the model’s responses. By doing so, we aimed to obtain more deterministic and predictable responses for each experiment. The pipeline comprises two main components: Prompt Formulation and Classification. The Prompt Formulation phase involves crafting guidelines and prompts, wherein GPT analyzes the support set to identify common characteristics within each class (i.e., intrinsic and non-intrinsic bugs) based on concise and specific instructions. The extracted characteristics, along with the general definitions of intrinsic and non-intrinsic bugs, serve as guidelines for the task context. These guidelines are formulated as follows:

$$guideline = def + char(I) + char(NI)$$

where *def* is the initial definition of the two classes, *char* represents characteristics, thus resulting in *char(I)* meaning characteristics of intrinsic bugs, and *char(NI)* meaning characteristics of non-intrinsic bugs. Additionally, GPT evaluates each bug report in the support set and assigns scores from 0 to 1 based on the provided guidelines, where 1 indicates that the bug report text perfectly represents the given bug label, whereas 0 indicates that it is the worst example of that bug label. Subsequently, the top-scoring instances, corresponding to the number of shots, are selected as representative examples.

$$example_s(I) = \operatorname{argmax}_{bug_report}(scores \forall label = "intrinsic")$$

$$example_s(NI) = \operatorname{argmax}_{bug_report}(scores \forall label = "non - intrinsic")$$

Finally, the prompt for each n^{th} shot experiment integrates the guideline and top-scoring examples for each label, which can be represented as:

$$prompt = guideline + \sum_{s=0}^{s=n} example_s(I) + \sum_{s=0}^{s=n} example_s(NI)$$

where n is the number of shots and s goes from 0 to n . The top-scoring examples are concatenated together until the number of shots is met.

6.3 Experimental Results

In this section, we present the results yielded from implementing a novel few-shot prompt engineering approach aimed at mitigating the issues caused

6.3. EXPERIMENTAL RESULTS

by a low-resource dataset. The findings presented here offer valuable insights into the performance and effectiveness of this new approach that has yet to be adopted in the bug classification problem tasks but has immense potential considering the latest developments in LLMs. In this section, we aim to answer the research question **To what extent does few-shot prompt engineering prove effective in classifying intrinsic bugs in the context of low-resource dataset?**

Motivation: Comparing the results from the previous experiments, we believe there might be room for improvement. The dataset has 1,880 bug records to use for training and testing purposes, which is not substantial enough for training sophisticated deep learning models or fine-tuning LLMs. Moreover, there is a significant class imbalance in the dataset, which might hinder the models from learning the patterns of the minority class (non-intrinsic bugs). The current trend of few-shot learning and prompt engineering tackle similar issues [4, 42] utilizing a general knowledge base accompanied by specific context and examples to extend the knowledge of LLMs, potentially being an alternative to classic AI techniques that have been used in bug classification so far. Therefore, we hypothesize that using few-shot prompt engineering can show comparable performance as the ML models, if not better, in accomplishing the same task using fewer data examples.

Approach: The implementation uses the GPT-3.5-turbo model to perform the preliminary experiments to test the efficacy of few-shot prompt engineering in solving the bug classification task for intrinsic and non-intrinsic bugs. The overall pipeline has two main components: prompt formulation and classification modules. Both involve the use of several prompt instructions as explained in Section 6.2. The overall method has been depicted in Figure 6.1.

The final prompts were selected after multiple iterations of various prompt messages that can be found in our replication package ²⁰. The prompt used in the Prompt Formulation process to fetch the common characteristics of intrinsic and non-intrinsic each to help provide the appropriate context is given below.

²⁰https://osf.io/c89tn/?view_only=8ddc37ec84cf4e83bac1ab2e209037d2

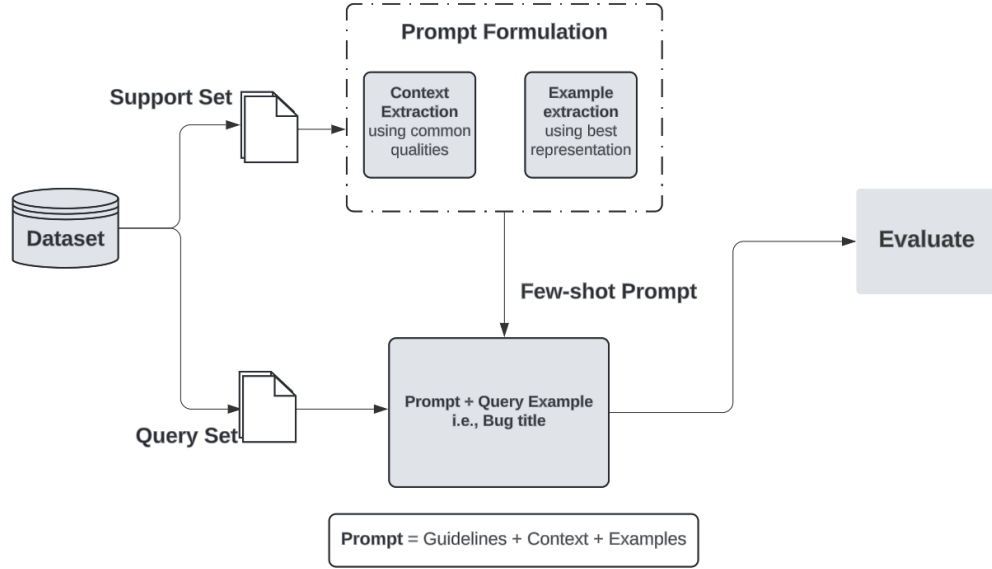


Figure 6.1: Experiment Setup of RQ3

Prompt: Look at the examples belonging to the same class, i.e., either intrinsic or non-intrinsic. Can you find the common qualities and patterns among the bug report text in each class and report it in bullet points for each class? Please keep your message as concise and to the point as possible.

The common qualities generated out of the support set by GPT are listed for each class.

Intrinsic Bugs:

- These bugs are related to the core functionality of the OpenStack components.
- They involve issues with the behavior, logic, or features of the OpenStack services.
- Many of these bugs are related to specific drivers or plugins used within OpenStack.

6.3. EXPERIMENTAL RESULTS

- The bugs often involve errors in handling data, configuration, or interactions between different components.
- Some bugs are related to specific API calls, methods, or functions within the OpenStack services.

Non-Intrinsic Bugs:

- These bugs are related to external integrations, plugins, or drivers used with OpenStack.
- They involve issues with third-party services, drivers, or plugins interacting with OpenStack components.
- Many of these bugs are specific to certain drivers or plugins used in conjunction with OpenStack services.
- The bugs often involve errors in communication, data exchange, or compatibility between OpenStack and external components.
- Some bugs are related to specific functionalities or features provided by third-party services integrated with OpenStack.

The final prompt resulting from the Prompt Formulation process is given below.

Prompt: Given below are some guidelines and characteristics of two types of bugs, intrinsic and non-intrinsic. Please read them as a software developer who has worked in open-source software projects. Intrinsic bugs are bugs caused by internal code evolution within the OpenStack repositories and internal changes that caused failure reported as bugs. Whereas Non-intrinsic bugs are those that are caused by external changes or external code evolution that affect the OpenStack repositories. // Generated guidelines//. Based on the guidelines and definitions of intrinsic, and non-intrinsic, could you please evaluate if the given bug description can most perfectly represent its class by scoring it from 0 to 1? Your response should be in a Python dictionary format with a floating score between 0 and 1 as the value for the key “score”, and the description of why the score was given for the key “evaldesc”. Ensure there should only be the dictionary object without any other information attached to it.

6.3. EXPERIMENTAL RESULTS

Here, the “Generated guidelines” represent the common characteristics extracted from GPT analysis. The guidelines specified for each class are inserted in place of that text. Finally, for the Classification module, we needed to design a query prompt, which was finalized as given below.

Prompt: Based on the examples, could you classify the following issue texts into intrinsic or non-intrinsic? Your response should be your predicted label which can only be one of two options (intrinsic, non-intrinsic), and the confidence interval between 1 and 100 with which you made that classification. The output should be comma-separated predicted label and confidence interval, and No other information should be present. An example of the expected output is of the format non-intrinsic,80.
Examples (Text, label)
Query text (Text)

In this prompt, the number of examples would depend on the number of shots in that experiment and the choice of bug report whose text and label were used was based on the top scores given to the bug reports in each class based on an analysis by GPT to fetch the best-representing instances from the dataset. Whereas the query text would contain the bug report text of an instance from the query set that was reserved from the dataset for querying and testing purposes.

Results: The results are detailed in Table 6.1. The performance metrics indicate notable variations across different shot configurations. For instance, with zero-shots, the model achieved a precision of 67%, recall of 76%, an F1 score of 71%, and an AUC-ROC score of 60%. Certain metrics improved as the number of shots increased, while others experienced fluctuations. Notably, the model attained its highest F1 score of 73% with one-shot and 10-shot experiments, demonstrating an almost cyclical effect in terms of the F1 score. However, precision and recall values showed varying trends across different shot configurations, suggesting the need for careful consideration in selecting the optimal number of shots. AUC-ROC displayed a similar pattern of fluctuation across different shot configurations, reaching a peak of 62.58% at 10 shots with a slight decrease at 20 shots scoring 60.76%.

Overall, the results underscore the potential of Few-Shot Prompt Engineering with the GPT-3.5-turbo model in accurately classifying bugs into intrinsic and non-intrinsic categories, with performance varying based on

6.4. DISCUSSION

Table 6.1: Results of Few-Shot Prompt Engineering to Classify Intrinsic and Non-Intrinsic Bugs

Shots	Precision	Recall	F1 Score	AUC-ROC
0	67%	76%	71%	60%
1	63%	87%	73%	54.99%
3	68%	62%	65%	59.79%
5	68%	65%	67%	60.06%
8	69%	58%	63%	60.10%
10	69%	78%	73%	62.58%
20	68%	70%	69%	60.76%

the number of shots. In this study, the best F1 and AUC-ROC scores are achieved with 10 shots reaching 73% and 62.58% respectively.

6.4 Discussion

The outcomes of this study present interesting insights into the performance of various shot experiments. Notably, the 10-shot prompt-engineering experiment yielded the highest F1 score of 73%, matching the F1 of the 1-shot experiment and surpassing other shot variations in terms of AUC-ROC, achieving a score of 62.58%. These results become particularly intriguing when compared to the preceding 8-shot and subsequent 20-shot experiments. A significant increase of 10 points in F1 score is observed from 8 shots to 10 shots, followed by a decrease to 69% F1 at 20 shots. Similarly, a parabolic trend is observed in the AUC-ROC score, with a slight increase from 8 shots to 10 shots (60.10% to 62.58%) before decreasing back to 60.76% at 20 shots. Interestingly, these patterns resonate with findings from the study by Nashid et al. [42], which explored few-shot learning in the context of test assertion generation and program repair. Nashid et al. [42] reported optimal results at 8 shots before observing a slight decline in performance at 20 shots. Their observations suggest that increasing the number of demonstrations may not necessarily improve prediction accuracy. We conjecture that it could potentially be due to the complexity introduced by additional patterns or trends in the learning process. It remains an intriguing prospect to investigate whether such trends follow a cyclic pattern across a wider range of shot experiments or if an optimal shot number exists beyond which performance diminishes consistently.

Additionally, the usage of a general-purpose LLM provided further per-

formance enhancements despite the few-shot setting. Despite this progress, experiments with LLMs did not surpass the performance of ML-based classifiers. Also, there are other LLMs that are open-source such as Llama-2 [59] and Llama-3 ²¹, and that can provide more cost benefits than GPT-3 as users do not have to pay to use them.

It is worthy to note that we also performed similar experiments of few-shot prompt engineering experiments using GPT-4-turbo hoping for significantly better performance than GPT-3.5-turbo, however, GPT-4 yielded worse results than GPT-3 in our case. This could be attributed to the specific models we used, as GPT-4-turbo does not represent all GPT-4 models. It is worth noting that given the expenses linked to employing GPT-4 models, the rationale behind utilizing and exploring few-shot prompt engineering becomes questionable when conducting the n-shot experiments. This is primarily due to the potential scenario where the costs incurred from utilizing and experimenting with GPT-4 may surpass or equal the expenses associated with manually labeling additional data to address the constraints of a low-resource dataset. Therefore, we believe that careful consideration of cost-effectiveness is crucial when deciding between utilizing GPT-4 models and alternative approaches, particularly in low-resource data settings.

Curating bug datasets is an inherently resource-intensive process. This process involves specialized knowledge of the software system being studied, as well as manual effort to collect and preprocess the data. As a result, it can be costly in terms of both time and expertise. Therefore, the development of tools to assist in bug classification shows potential for help researchers who face these challenges. Moreover, our study also shows promising result when using GPT models. Hence, researchers can use or enhance our proposed approaches to automatically identify and flag intrinsic bugs based on bug report titles, descriptions, and bug fixing commits linked to bug reports.

6.5 Threats to Validity

External Validity The most limiting factor of using LLM in our experiments has been the unpredictable nature of instruction following by the LLM. Although we set the temperature to 0, we observed some iterations in which the responses were random and the model did not follow the prompt instructions displaying erroneous behavior. This resulted in significant variance in some experiments that may have introduced some external threats to the pipeline.

²¹<https://ai.meta.com/blog/meta-llama-3/>

6.6 Conclusion and Future Work

In the future, researchers can explore different LLMs like Llama variants, GPT-4, and other contemporary models to improve bug classification accuracy. Moreover, there are several possibilities of experimentation that can be performed to analyze the performance of few-shot prompt engineering, for instance, using more factors to select the best-representing examples from the dataset. It would be interesting to study if enforcing some rules to include examples of varying projects could increase the performance of our current pipeline. However, amidst all the experiments with LLMs that have a paywall, it is important to be conscious of the cost of the experiments that are being conducted. The research and experiments with few-shot prompt engineering would defeat their own purpose if running the pipeline to classify bugs ends up costing more than labelling more dataset manually with human precision and consensus. Despite this, we predict exciting new techniques that can be applied in classifying bugs with the advancement of LLMs.

Chapter 7

Conclusion

Research dedicated to analyzing, predicting, and preventing bugs holds immense significance, as it underpins the advancement of the software development process. Comprehensive understanding of bugs is pivotal in this context. Such comprehension encompasses various methods, including analyzing textual elements such as reporters' entries in bug reports, which comprise natural language text, code snippets, and error logs. Additionally, understanding bugs entails scrutinizing bug-fixing code changes and their attributes. Bug classification emerges as a crucial endeavor in better categorizing bugs and unraveling their underlying nature. Hence, automating bug classification processes holds the promise of significantly enhancing the efficiency of bug triage procedures and augmenting the performance of bug prediction models.

In our research endeavor, we developed the first novel automatic bug classification system aimed at categorizing intrinsic and non-intrinsic bugs. Leveraging textual information extracted from bug reports, we trained NLP and ML models for this purpose. Subsequently, we sought to enhance the classifier's performance by incorporating additional non-textual features while maintaining a similar pipeline to the initial classifier. Furthermore, we introduced a novel tool utilizing title text in conjunction with LLMs through a few-shot prompt engineering approach to classify intrinsic and non-intrinsic bugs. This innovation showcased the potential of employing such methodologies in bug classification tasks where labeled data is scarce.

The realm of bug classification, particularly concerning the differentiation between intrinsic and non-intrinsic bugs, harbors considerable potential for further exploration. We encourage researchers to delve into diverse avenues, methodologies, and empirical analyses within this domain. By collectively advancing our understanding and methodologies, we can strive towards refining bug classification practices, ultimately reaching a level of confidence that can obviate the need for manual classification processes.

Bibliography

- [1] *Speech and Language Processing. logistic regression.* <https://web.stanford.edu/~jurafsky/slp3/5.pdf>. Accessed: 2023-07-09. → pages 23
- [2] P. AFRIC, D. VUKADIN, M. SILIC, AND G. DELAC, *Empirical study: How issue classification influences software defect prediction*, IEEE Access, 11 (2023), pp. 11732–11748. → pages 11, 12, 13
- [3] H. A. AHMED, N. Z. BAWANY, AND J. A. SHAMSI, *Capbug-a framework for automatic bug categorization and prioritization using nlp and machine learning algorithms*, IEEE Access, 9 (2021), pp. 50496–50512. → pages 2, 3, 11, 12, 14, 15, 21, 22, 23, 42
- [4] T. AHMED, K. S. PAI, P. DEVANBU, AND E. T. BARR, *Automatic semantic augmentation of language model prompts (for code summarization)*, 2024. → pages 56, 58
- [5] F. AKHBARDEH, C. O. ALM, M. ZAMPIERI, AND T. DESELL, *Handling extreme class imbalance in technical logbook datasets*, in Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), Association for Computational Linguistics, 2021. → pages 37
- [6] E. AKTAS, E. CAKMAK, M. INAN, AND C. YILMAZ, *Issue report validation in an industrial context*, in Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE '23, ACM, Nov. 2023. → pages 2, 11, 12, 14, 42
- [7] I. ALAZZAM, A. ALEROUD, Z. AL LATIFAH, AND G. KARABATIS, *Automatic bug triage in software systems using graph neighborhood relations for feature augmentation*, IEEE Transactions on Computational Social Systems, 7 (2020), pp. 1288–1303. → pages 2, 3, 11, 12, 14, 21, 22, 23, 37, 42

- [8] A. APRIL AND A. ABRAN, *Maintenance Issues and Related Management Approaches*, 2008, pp. 1–39. → pages 1
- [9] L. BAO, X. XIA, A. E. HASSAN, AND X. YANG, *V-szz: automatic identification of version ranges affected by cve vulnerabilities*, in Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 2352–2364. → pages 3
- [10] J. BERGSTRA AND Y. BENGIO, *Random search for hyper-parameter optimization*, J. Mach. Learn. Res., 13 (2012), p. 281–305. → pages 23, 42
- [11] P. BHANDARI AND G. RODRÍGUEZ-PÉREZ, *Buggin: Automatic intrinsic bugs classification model using nlp and ml*, in Proceedings of the 19th International Conference on Predictive Models and Data Analytics in Software Engineering, PROMISE 2023, New York, NY, USA, 2023, Association for Computing Machinery, p. 2–11. → pages 4, 18, 37, 38, 41, 42, 43, 44, 49, 50, 51
- [12] S. BHARADWAJ AND T. KADAM, *Github issue classification using bert-style models*, in 2022 IEEE/ACM 1st International Workshop on Natural Language-Based Software Engineering (NLBSE), 2022, pp. 40–43. → pages 2, 3, 11, 23, 33
- [13] L. BREIMAN, *Random forests*, Machine Learning, 45 (2001), pp. 5–32. → pages 23
- [14] T. B. BROWN, B. MANN, N. RYDER, M. SUBBIAH, J. KAPLAN, P. DHARIWAL, A. NEELAKANTAN, P. SHYAM, G. SASTRY, A. ASKELL, S. AGARWAL, A. HERBERT-VOSS, G. KRUEGER, T. HENIGHAN, R. CHILD, A. RAMESH, D. M. ZIEGLER, J. WU, C. WINTER, C. HESSE, M. CHEN, E. SIGLER, M. LITWIN, S. GRAY, B. CHESS, J. CLARK, C. BERNER, S. MCCANDLISH, A. RADFORD, I. SUTSKEVER, AND D. AMODEI, *Language models are few-shot learners*, 2020. → pages 16
- [15] G. CATOLINO, F. PALOMBA, A. ZAIDMAN, AND F. FERRUCCI, *Not all bugs are the same: Understanding, characterizing, and classifying the root cause of bugs*, 2019. → pages 3, 11, 12, 15, 21, 22, 23, 31, 37, 42
- [16] N. V. CHAWLA, K. W. BOWYER, L. O. HALL, AND W. P. KEGELMEYER, *SMOTE: Synthetic minority over-sampling technique*,

- Journal of Artificial Intelligence Research, 16 (2002), pp. 321–357. → pages 22
- [17] G. COLAVITO, F. LANUBILE, N. NOVIELLI, AND L. QUARANTA, *Impact of data quality for automatic issue classification using pre-trained language models*, Journal of Systems and Software, 210 (2024), p. 111838. → pages 11, 15
 - [18] N. CRISTIANINI AND J. SHAWE-TAYLOR, *An Introduction to Support Vector Machines and Other Kernel-based Learning Methods*, Cambridge University Press, Mar. 2000. → pages 22
 - [19] J. C. B. CRUZ AND C. CHENG, *Improving large-scale language models and resources for filipino*, 2021. → pages 16
 - [20] P. CUNNINGHAM AND S. J. DELANY, *k-nearest neighbour classifiers - a tutorial*, ACM Computing Surveys, 54 (2021), pp. 1–25. → pages 23
 - [21] D. A. DA COSTA, S. MCINTOSH, W. SHANG, U. KULESZA, R. COELHO, AND A. E. HASSAN, *A framework for evaluating the results of the szz approach for identifying bug-introducing changes*, IEEE Transactions on Software Engineering, 43 (2016), pp. 641–657. → pages 33
 - [22] J. DEVLIN, M.-W. CHANG, K. LEE, AND K. TOUTANOVA, *Bert: Pre-training of deep bidirectional transformers for language understanding*, 2019. → pages 21
 - [23] A. DI SORBO, F. ZAMPETTI, A. VISAGGIO, M. DI PENTA, AND S. PANICHELLA, *Automated identification and qualitative characterization of safety concerns reported in uav software platforms*, ACM Trans. Softw. Eng. Methodol., 32 (2023). → pages 2, 4, 11, 12, 14, 22, 37
 - [24] Q. FAN, Y. YU, G. YIN, T. WANG, AND H. WANG, *Where is the road for issue reports classification based on text mining?*, in 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), 2017, pp. 121–130. → pages 3, 11, 12, 13, 21, 22, 33
 - [25] Y. FAN, X. XIA, D. A. DA COSTA, D. LO, A. E. HASSAN, AND S. LI, *The impact of mislabeled changes by szz on just-in-time defect prediction*, IEEE transactions on software engineering, 47 (2019), pp. 1559–1586. → pages 33

- [26] W. FU, T. MENZIES, AND X. SHEN, *Tuning for software analytics: Is it really necessary?*, Information and Software Technology, 76 (2016), pp. 135–146. → pages 34
- [27] S. GUO, R. CHEN, M. WEI, H. LI, AND Y. LIU, *Ensemble data reduction techniques and multi-rsmote via fuzzy integral for bug report classification*, IEEE Access, 6 (2018), pp. 45934–45950. → pages 2, 11, 12, 15, 23, 37
- [28] S. K. HAMOURI, R. A. SHATNAWI, O. ALZOUBI, A. MIGDADY, AND M. B. YASSEIN, *Predicting bug severity using machine learning and ensemble learning techniques*, in 2023 14th International Conference on Information and Communication Systems (ICICS), 2023, pp. 1–6. → pages 4, 22, 37
- [29] J. HEO AND S. LEE, *An empirical study on the performance of individual issue label prediction*, in 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR), 2023, pp. 228–233. → pages 11, 12, 15
- [30] S. HERBOLD, A. TRAUTSCH, B. LEDEL, A. AGHAMOHAMMADI, T. A. GHALEB, K. K. CHAHAL, T. BOSSENMAIER, B. NAGARIA, P. MAKEDONSKI, M. N. AHMADABADI, K. SZABADOS, H. SPIEKER, M. MADEJA, N. HOY, V. LENARDUZZI, S. WANG, G. RODRÍGUEZ-PÉREZ, R. COLOMO-PALACIOS, R. VERDECCHIA, P. SINGH, Y. QIN, D. CHAKROBORTI, W. DAVIS, V. WALUNJ, H. WU, D. MARCILIO, O. ALAM, A. ALDAEEJ, I. AMIT, B. TURHAN, S. EISMANN, A.-K. WICKERT, I. MALAVOLTA, M. SULÍR, F. FARD, A. Z. HENLEY, S. KOURTZANIDIS, E. TUZUN, C. TREUDE, S. M. SHAMASBI, I. PASHCHENKO, M. WYRICH, J. DAVIS, A. SEREBRENIK, E. ALBRECHT, E. U. AKTAS, D. STRÜBER, AND J. ERBEL, *A fine-grained data set and analysis of tangling in bug fixing commits*, Empirical Software Engineering, 27 (2022). → pages 53
- [31] M. IZADI, K. AKBARI, AND A. HEYDARNOORI, *Predicting the objective and priority of issue reports in a cross project context*, CoRR, abs/2012.10951 (2020). → pages 2, 3, 11, 12, 14, 15, 21, 22, 23, 42
- [32] A. JOULIN, E. GRAVE, P. BOJANOWSKI, AND T. MIKOLOV, *Bag of tricks for efficient text classification*, 2016. → pages 12, 13, 14, 15

- [33] R. KALLIS, A. DI SORBO, G. CANFORA, AND S. PANICHELLA, *Ticket tagger: Machine learning driven issue classification*, in 2019 IEEE International Conference on Software Maintenance and Evolution (IC-SME), 2019, pp. 406–409. → pages 2, 11, 12, 15, 23, 37
- [34] Y. KAMEI, E. SHIHAB, B. ADAMS, A. E. HASSAN, A. MOCKUS, A. SINHA, AND N. UBAYASHI, *A large-scale empirical study of just-in-time quality assurance*, IEEE Transactions on Software Engineering, 39 (2013), pp. 757–773. → pages 3
- [35] Z. LI, M. PAN, Y. PEI, T. ZHANG, L. WANG, AND X. LI, *Deeplabel: Automated issue classification for issue tracking systems*, in Proceedings of the 13th Asia-Pacific Symposium on Internetware, Internetware ’22, New York, NY, USA, 2022, Association for Computing Machinery, p. 231–241. → pages 2, 3, 11, 12, 23, 32, 33
- [36] Y. LIU, M. OTT, N. GOYAL, J. DU, M. JOSHI, D. CHEN, O. LEVY, M. LEWIS, L. ZETTLEMOYER, AND V. STOYANOV, *Roberta: A robustly optimized bert pretraining approach*, 2019. → pages 13, 16
- [37] L. LOUKAS, I. STOGIANNIDIS, P. MALAKASIOTIS, AND S. VASSOS, *Breaking the bank with chatgpt: Few-shot text classification for finance*, 2023. → pages 16, 55
- [38] L. LOWPHANSIRIKUL, C. POLPANUMAS, N. JANTRAKULCHAI, AND S. NUTANONG, *Wangchanberta: Pretraining transformer-based thai language models*, 2021. → pages 16
- [39] S. MCINTOSH AND Y. KAMEI, *Are fix-inducing changes a moving target? a longitudinal case study of just-in-time defect prediction*, in Proceedings of the 40th International Conference on Software Engineering, 2018, pp. 560–560. → pages ix, 39, 40
- [40] T. MIKOLOV, I. SUTSKEVER, K. CHEN, G. CORRADO, AND J. DEAN, *Distributed representations of words and phrases and their compositionality*, 2013. → pages 13
- [41] H. MOHSIN, C. SHI, S. HAO, AND H. JIANG, *Span: A self-paced association augmentation and node embedding-based model for software bug classification and assignment*, Knowledge-Based Systems, 236 (2022), p. 107711. → pages 37

- [42] N. NASHID, M. SINTAHA, AND A. MESBAH, *Retrieval-based prompt selection for code-related few-shot learning*, in 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023, pp. 2450–2462. → pages 56, 58, 62
- [43] E. C. NETO, D. A. DA COSTA, AND U. KULESZA, *The impact of refactoring changes on the szz algorithm: An empirical study*, in 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2018, pp. 380–390. → pages 3, 33
- [44] Z. NI, B. LI, X. SUN, T. CHEN, B. TANG, AND X. SHI, *Analyzing bug fix for automatic bug cause classification*, Journal of Systems and Software, 163 (2020), p. 110538. → pages 3, 11, 23, 37
- [45] S. PATIL AND B. RAVINDRAN, *Predicting software defect type using concept-based classification*, Empirical Software Engineering, 25 (2020), p. 1341–1378. → pages 11, 21, 23
- [46] N. PINGCLASAI, H. HATA, AND K.-I. MATSUMOTO, *Classifying bug reports to bugs and other requests using topic modeling*, in 2013 20th Asia-Pacific Software Engineering Conference (APSEC), vol. 2, 2013, pp. 13–18. → pages 13
- [47] H. QIN AND X. SUN, *Classifying bug reports into bugs and non-bugs using lstm*, in Proceedings of the 10th Asia-Pacific Symposium on Internetware, Internetware '18, New York, NY, USA, 2018, Association for Computing Machinery. → pages 2, 3, 11, 12, 13, 23
- [48] J. R. QUINLAN, *Induction of decision trees*, Machine Learning, 1 (1986), pp. 81–106. → pages 23
- [49] N. REIMERS AND I. GUREVYCH, *Sentence-bert: Sentence embeddings using siamese bert-networks*, 2019. → pages 15
- [50] C. REZK, Y. KAMEI, AND S. MCINTOSH, *The ghost commit problem when identifying fix-inducing changes: An empirical study of apache projects*, IEEE Transactions on Software Engineering, 48 (2021), pp. 3297–3309. → pages 33
- [51] G. RODRÍGUEZ-PÉREZ, G. ROBLES, A. SEREBRENIK, A. ZAIDMAN, D. M. GERMÁN, AND J. M. GONZALEZ-BARAHONA, *How bugs are born: a model to identify how bugs are introduced in software components*, Empirical Software Engineering, 25 (2020), pp. 1294–1340. → pages 2, 3

- [52] G. RODRÍGUEZ-PÉREZ, A. ZAIDMAN, A. SEREBRENİK, G. ROBLES, AND J. M. GONZÁLEZ-BARAHONA, *What if a bug has a different origin? making sense of bugs without an explicit bug introducing change*, (2018). → pages 2, 3, 8, 9
- [53] G. RODRÍGUEZ-PÉREZ, M. NAGAPPAN, AND G. ROBLES, *Watch out for extrinsic bugs! a case study of their impact in just-in-time bug prediction models on the openstack project*, IEEE Transactions on Software Engineering, 48 (2022), pp. 1400–1416. → pages 3, 19, 20, 33, 34, 39, 43, 49, 52
- [54] E. SAHAL AND A. TOSUN, *Identifying bug-inducing changes for code additions*, in Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, 2018, pp. 1–2. → pages 33
- [55] J. ŚLIWERSKI, T. ZIMMERMANN, AND A. ZELLER, *When do changes induce fixes?*, ACM sigsoft software engineering notes, 30 (2005), pp. 1–5. → pages 3
- [56] C. TANTITHAMTHAVORN, S. MCINTOSH, A. E. HASSAN, AND K. MATSUMOTO, *The impact of automated parameter optimization on defect prediction models*, IEEE Transactions on Software Engineering, 45 (2018), pp. 683–711. → pages 34
- [57] P. TERDCHANAKUL, H. HATA, P. PHANNACHITTA, AND K. MATSUMOTO, *Bug or not? bug report classification using n-gram idf*, in 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), 2017, pp. 534–538. → pages 11, 12, 13, 23
- [58] T. THAMINKAEW, P. LERTVITTAYAKUMJORN, AND P. VATEEKUL, *Prompt-based label-aware framework for few-shot multi-label text classification*, IEEE Access, 12 (2024), pp. 28310–28322. → pages 16, 55
- [59] H. TOUVRON, L. MARTIN, K. STONE, P. ALBERT, A. ALMAHAIRI, Y. BABAEI, N. BASHLYKOV, S. BATRA, P. BHARGAVA, S. BHOSALE, D. BIKEL, L. BLECHER, C. C. FERRER, M. CHEN, G. CUCURULL, D. ESIÖBU, J. FERNANDES, J. FU, W. FU, B. FULLER, C. GAO, V. GOSWAMI, N. GOYAL, A. HARTSHORN, S. HOSSEINI, R. HOU, H. INAN, M. KARDAS, V. KERKEZ, M. KHABSA, I. KLOUMANN, A. KORENEV, P. S. KOURA, M.-A. LACHAUX, T. LAVRIL, J. LEE, D. LISKOVICH, Y. LU, Y. MAO, X. MARTINET, T. MIHAYLOV,

- P. MISHRA, I. MOLYBOG, Y. NIE, A. POULTON, J. REIZENSTEIN, R. RUNGTA, K. SALADI, A. SCHELTEN, R. SILVA, E. M. SMITH, R. SUBRAMANIAN, X. E. TAN, B. TANG, R. TAYLOR, A. WILLIAMS, J. X. KUAN, P. XU, Z. YAN, I. ZAROV, Y. ZHANG, A. FAN, M. KAMBADUR, S. NARANG, A. RODRIGUEZ, R. STOJNIC, S. EDUNOV, AND T. SCIALOM, *Llama 2: Open foundation and fine-tuned chat models*, 2023. → pages 63
- [60] A. TRAUTSCH AND S. HERBOLD, *Predicting issue types with sebert*, in Proceedings of the 1st International Workshop on Natural Language-based Software Engineering, 2022, pp. 37–39. → pages 42
- [61] W. UThER, D. MLADENIĆ, M. CIARAMITA, B. BERENDT, A. KOŁCZ, M. GROBELNIK, D. MLADENIĆ, M. WITBROCK, J. RISCH, S. BOHN, S. POTEET, A. KAO, L. QUACH, J. WU, E. KEOGH, R. MIKKULAINEN, P. FLENER, U. SCHMID, F. ZHENG, G. I. WEBB, AND S. NIJSSEN, *Tf-idf*, 2011. → pages 18, 37, 52
- [62] J. VON DER MOSEL, A. TRAUTSCH, AND S. HERBOLD, *On the validity of pre-trained transformers for natural language processing in the software engineering domain*, IEEE Transactions on Software Engineering, 49 (2023), pp. 1487–1507. → pages 18, 21, 37, 41, 52
- [63] C. WOHLIN, P. RUNESON, M. HÖST, M. C. OHLSSON, B. REGNELL, AND A. WESSLÉN, *Experimentation in software engineering*, Springer Science & Business Media, 2012. → pages 34, 52
- [64] Y. ZENG, Y. YU, Q. FAN, X. ZHANG, T. WANG, G. YIN, AND H. WANG, *Cross-project issue classification based on ensemble modeling in a social coding world*, Neural Information Processing, (2018), p. 281–292. → pages 11, 12, 13, 21, 23, 37
- [65] Y. ZHOU, Y. TONG, R. GU, AND H. GALL, *Combining text mining and data mining for bug report classification*, in 2014 IEEE International Conference on Software Maintenance and Evolution, 2014, pp. 311–320. → pages 12, 13, 14, 37